KH정보교육원

데이터베이스 구현

목 차

Part 1. DBMS(Database Management System) 설치하기

Chapter 1. DBMS 종류 4

Chapter 2. ORACLE Database 11g XE DBMS 환경 구축하기 8

Chapter 3. DBMS 설치 확인하기 10

Part 2. 데이터베이스 생성하기

Chapter 4. 데이터베이스 사용자 생성하기 14

Chapter 5. 권한 설정하기 22

Chapter 6. DDL - TABLE 과 제약조건 33

Part 3. 데이터베이스 오브젝트 생성하기

Chapter 7. 데이터베이스 오브젝트 생성 계획하기 52

Chapter 8. 데이터베이스 오브젝트 생성하기 56

Chapter 9. 공공데이터 모델링하기 75

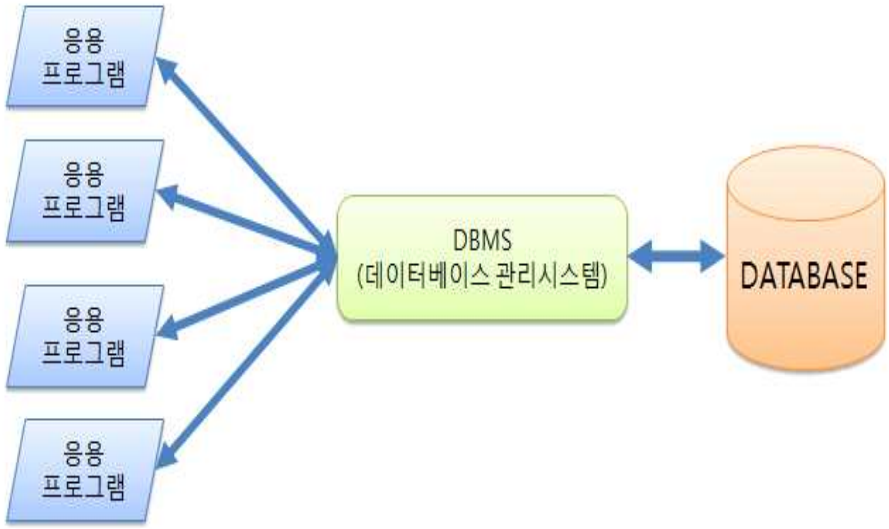
Part 1. DBMS(Database Management System) 설치하기

Chapter 1. DBMS 종류

1.1 데이터베이스 관리 시스템(DBMS)

- 응용 프로그램과 데이터베이스의 중재자로서 모든 응용프로그램들이 데이터베이스를 공용할 수 있게 관리해 주는 시스템

-



1.2 데이터베이스 관리 시스템(DBMS)의 종류

구 분	내 용
계층형 데이터베이스	구조를 트리형 구조로 정의하고 부모, 자식 형태를 갖는 구조이다. 자식은 하나의 부모만 가질 수 있다.
네트워크형 데이터베이스	트리형 구조를 갖지만 레코드 간 다양한 관계를 그물처럼 갖는 구조이다. 자식은 여러 부모를 가질 수 있다.
관계형 데이터베이스	키와 값들의 관계를 테이블 구조로 만든 것이다. 현재 다수의 데이터베이스가 관계형 데이터베이스를 적용하고 있다.
NoSQL 데이터베이스	관계형 데이터베이스이고 덜 제한적인 일관성 모델을 이용하는 데이터의 저장 및 검색을 위한 메커니즘을 제공한다. 빅데이터와 실시간 웹어플리케니션의 상업적 이용에 널리 쓰임.

일반적으로 RDBMS(Relationship DataBase Management System)를 많이 사용되고 있고, 우리가 배울 Oracle 역시 관계형 데이터베이스(RDBMS)이다.

1.3 관계형 데이터베이스(RDBMS) 특징

- 정보를 저장하기 위해 테이블을 이용하며, 행(row)과 열(column)로 구성된다.
- DB를 만들거나 이용하기가 다른 DBMS에 비해 쉽고, 확장이 용이하다.
- 처음 데이터베이스를 만든 후 관련되는 응용 프로그램들을 변경하지 않고 새로운 데이터 항목을 추가할 수 있다.

1.4 관계형 데이터베이스(RDBMS) 구조 및 용어



- **릴레이션(Relation)**

  : 테이블(표)을 의미한다.

- **릴레이션 인스턴스(Relation Instance)**

  : 데이터 개체를 구성하고 있는 속성(Attribute)들의 정의된 데이터 타입에 따라 구체적인 데이터 값을 갖고 있는 행들, 즉 저장되어 있는 테이블의 데이트 전체를 말한다.

- **릴레이션 스키마(Relation Schema)**

  : 속성(Attribute) 이름들의 집합을 말한다.

- **튜플(Tuple)**

  : 릴레이션을 구성하는 각각의 행으로 튜플은 속성의 모임으로 구성된다.

  튜플의 수를 카디날리티(cardinality) 또는 기수(cardinal number), 대응수라고 한다.

- **도메인(Domain)**

  : 하나의 애트리뷰트가 취할 수 있는 같은 타입의 원자값들의 집합, 실제 애트리뷰트 값이 나타날 때 그 값의 적절한지의 여부를 시스템이 검사할 때 도메인이 이용된다.

1.5 관계형 데이터베이스의 종류

DBMS 종류	회사명
Oracle	Oracle
MS-SQL Server	Microsoft
My-SQL	Oracle (subMicroSystem)
DB2	IBM
informix	IBM
Sybase	Sybase
Derby	APache
SQLite	Opensource

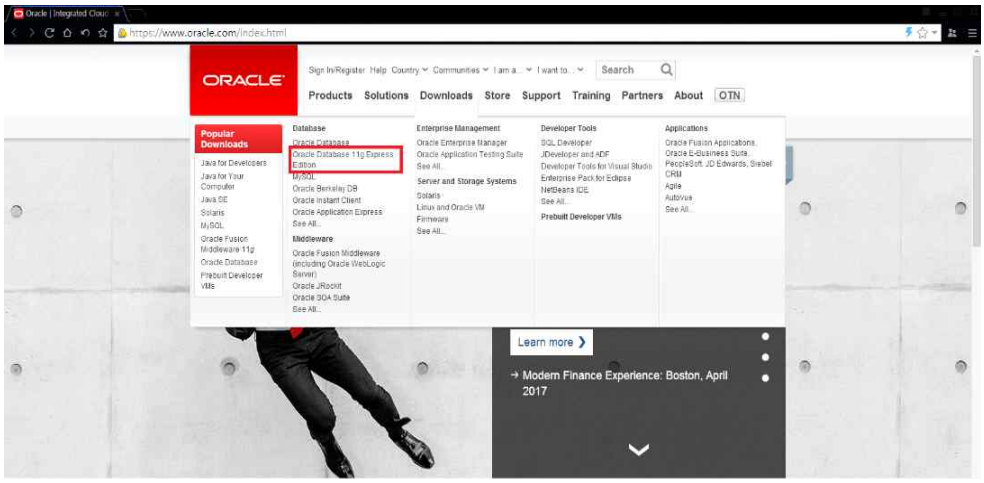
Chapter 2. ORACLE Database 11g XE DBMS 환경 구축하기

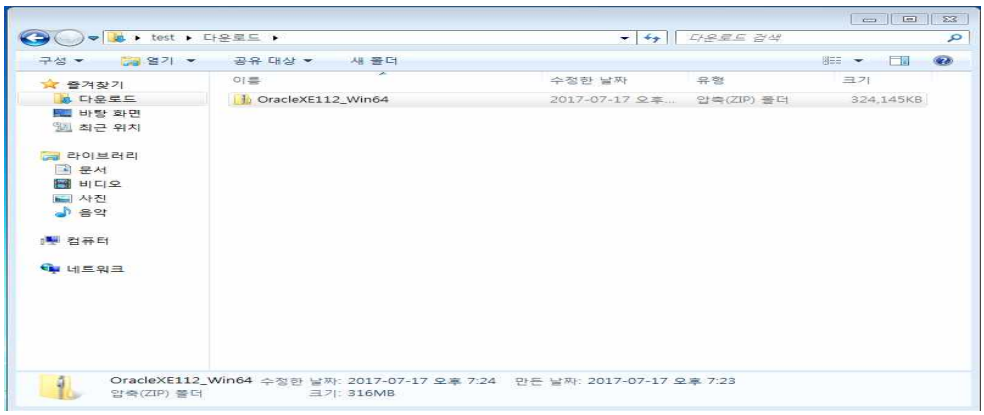
개발환경 구성

1.1. Oracle Database Express Edition 11g Release 2 설치

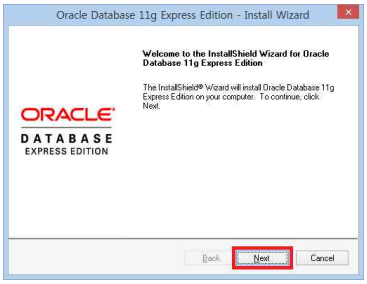
① www.oracle.com 접속

② Downloads - Oracle Database 11g Express Edition 다운

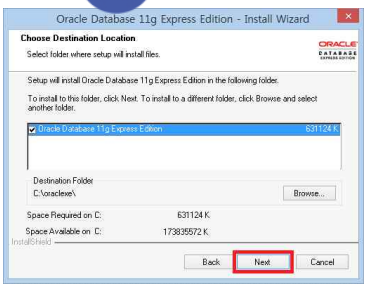


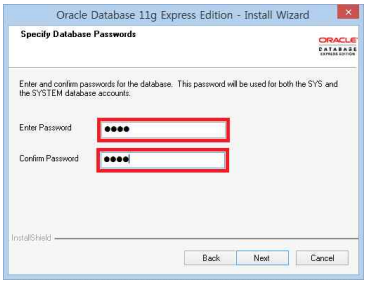


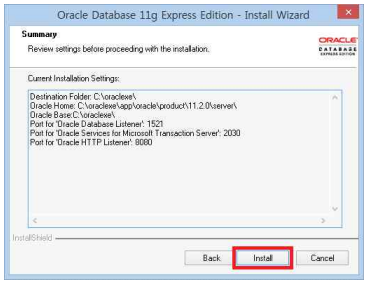
③ 설치

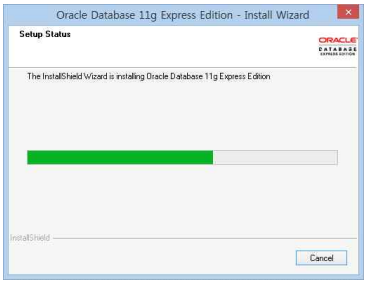




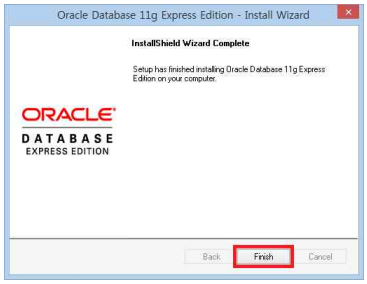








비밀번호는 최고 관리자 sys/system계정의 비빌번호로 oracle로 하겠다. 사용자가 하고 싶은 것으로 해도 되지만 비밀번호는 반드시 기억해야 한다. **Express Edition 모듈은 비밀번호를 잊어버리면 다시 설치해야 한다.**

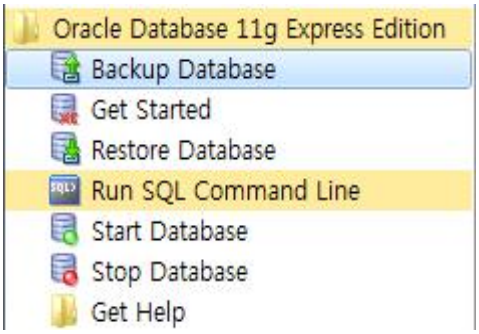


Chapter 3. DBMS 설치 확인하기

1. DBMS 설치 확인

1.1. Oracle Database Express Edition 11g Release 2 설치 확인

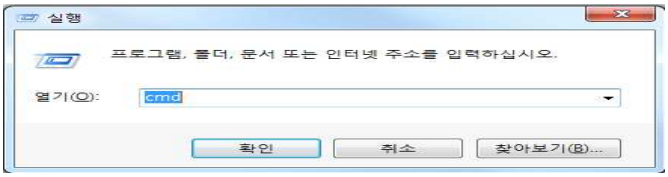
- SQL Plus를 이용해 Oracle 접속 확인



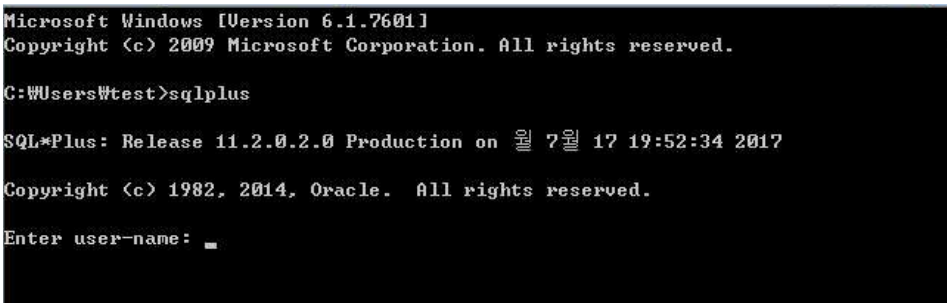
1.2. SQLPlus 실행

- SQL문을 실행시키고 결과를 볼 수 있도록 Oracle에서 제공하는 기본적인 Tool로서CMD 창에서 '**sqlplus**'를 입력하거나 'Run SQL Command Line'으로 실행 가능

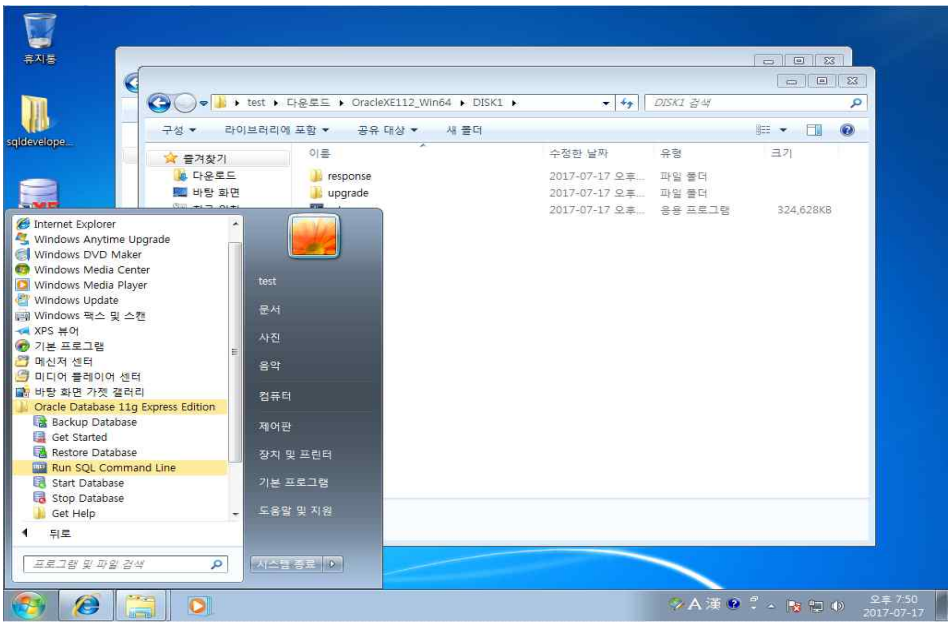
☞ CMD창 실행 : 윈도우키 + R 누르고 실행화면이 나오면 '**cmd**'누르고 엔터

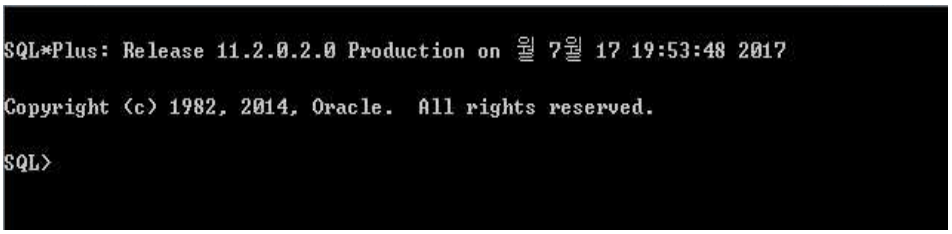


☞ Command Prompt 화면에서 **sqlplus** 입력하고 엔터



☞ 윈도우시작버튼 > 모든프로그램 > Oracle Datavase 11g Express Edition > Run SQL Command Line을 클릭한다.

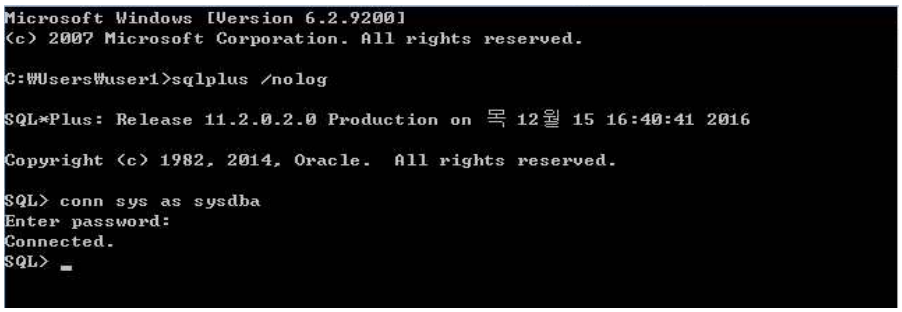




1.3. SQLPLUS 로그인하기

- Start Database 클릭
- sqlplus /nolog 실행

☞ login없이 database에 접급하는 명령어



- connect(conn도 사용가능) 사용자명 as 계정 해당권한(sysdba) 입력하여 DB에 연결

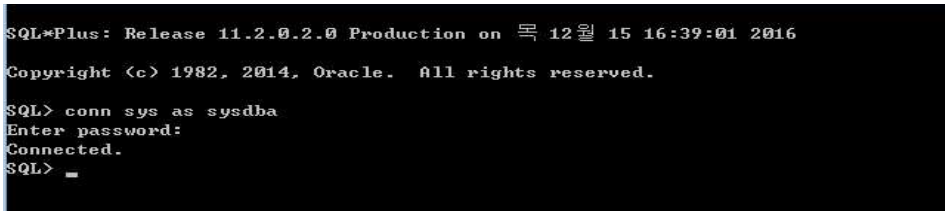
- password입력 ※ 설치시 설정한 비밀번호 입력.(예시 : oracle)

☞ 최고관리자 계정으로 로그인해보자.

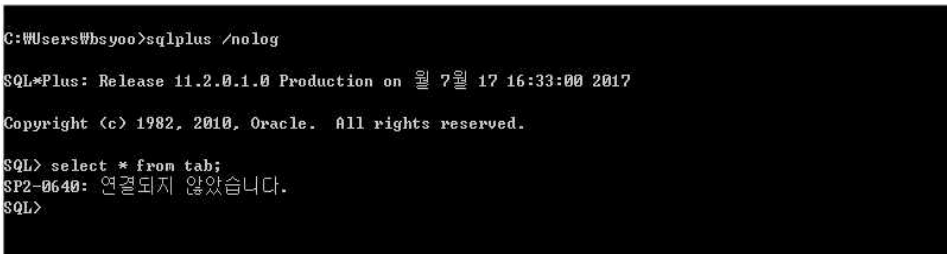
명령어 : conn sys as sysdba

Enter passworkd : 'oralce' 입력

☞ 비밀번호입력시 화면에 출력되지 않음



- DB연결하지 않고 즉 connect명령으로 접속하지 않고 DB조회(select문 작성시)하면 에러가 발생한다.



Part 2. 데이터베이스 생성하기

Chapter 4. 데이터베이스 사용자 생성하기

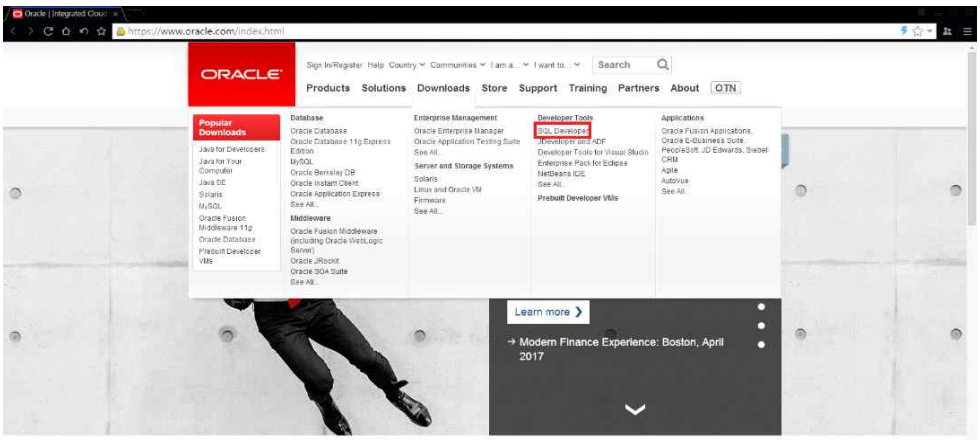
1. SQL Developer를 통한 사용자 계정 생성

1.1. SQL Developer 설치

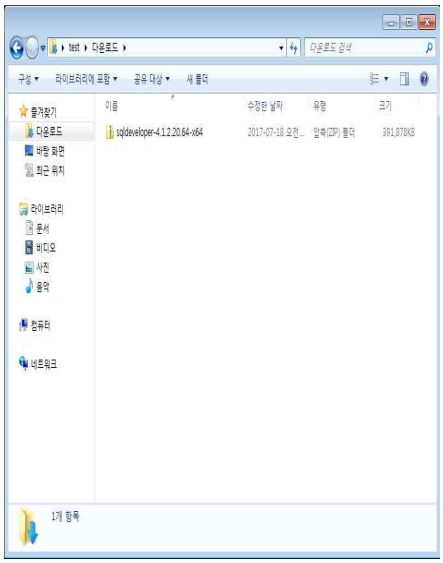
☞ **Java jre가 설치 되어 있어야 작동하므로 Java jre를 설치되어 있지 않으면 설치해야 한다. Java수업시간에 설치방법에 대해 하였기 때문에 설명하지 않겠다. JDK, JRE 아무거나 설치해도 상관없다.**

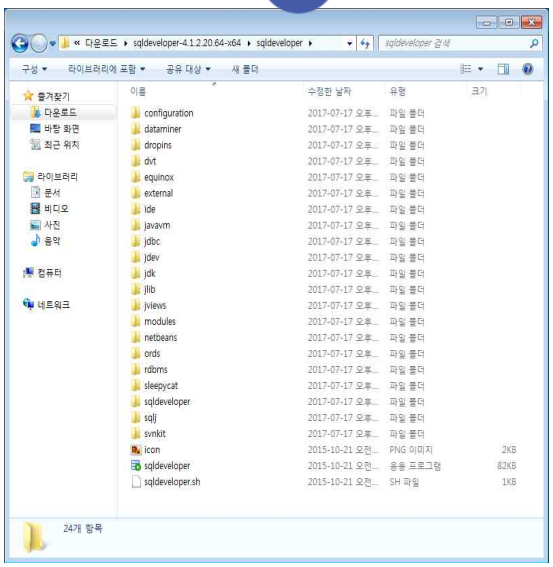
① www.oracle.com 에 접속

② Downloads - SQL Developer 다운



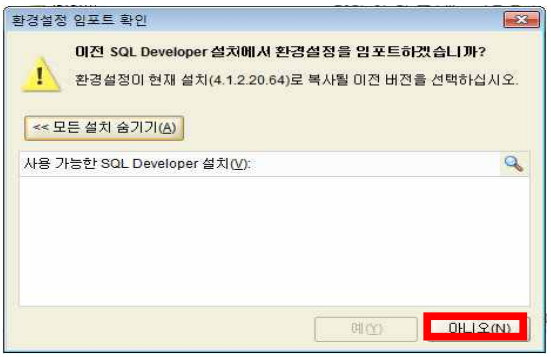
③ 압축 해제 후 바로 실행 가능





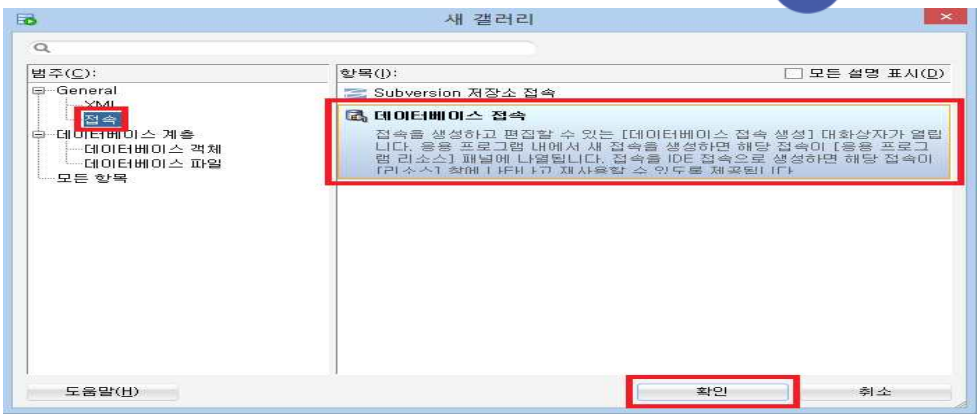
sqldeveloper파일 더블클릭하여 실행

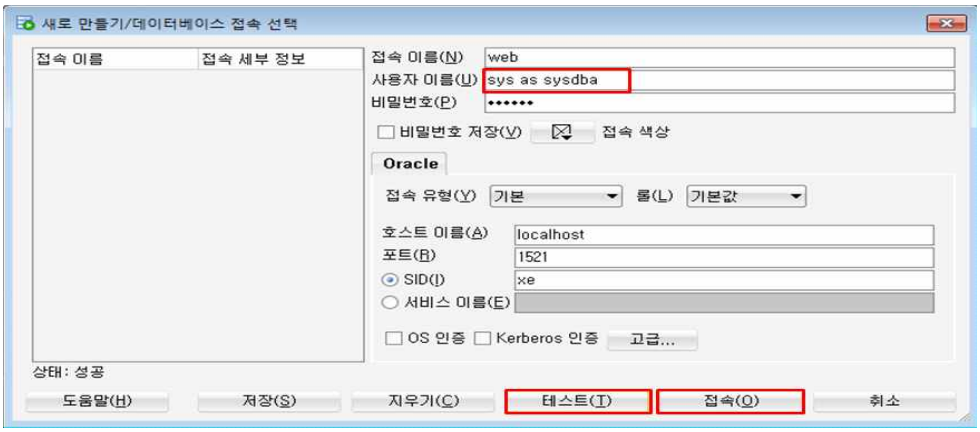






④ 파일 - 새로 만들기 - General - 접속 - 데이터베이스 접속



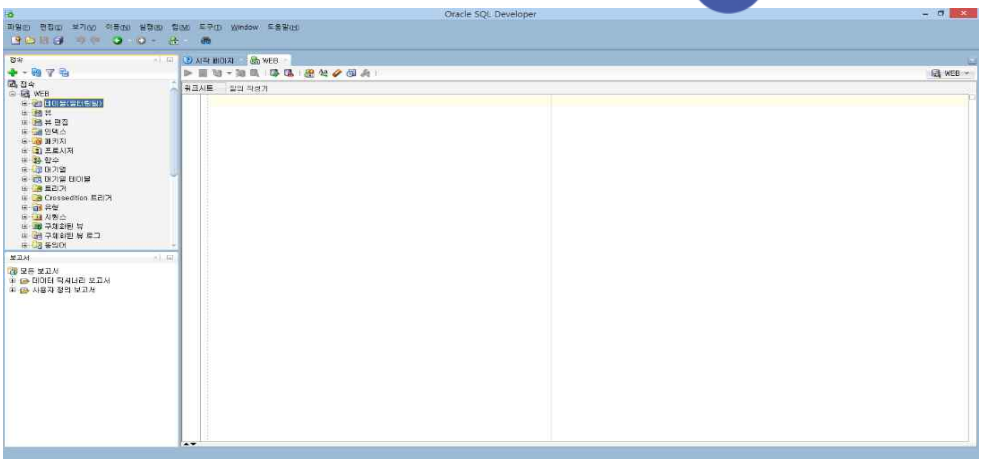


접속이름에는 연결이름 사용자가 정한 이름을 기재하고, 사용자 이름은 사용자 계정을 입력해야 하는데 예제에서는 sys(최고관리자)계정으로 접속 하여 뒤에 권한을 'as sysdba' 라고 하고 명시해줬다. 비밀번호는 초기에 설정한 비밀번호를 입력하면 된다.('oracle')

⑤ 정보 입력 후 테스트, 접속

- 접속 이름 : 원하는 이름으로 입력
- 사용자 이름 : 접속 테스트를 위해 'sys' 입력
- 비밀번호 : 데이터베이스 설치 시 입력한 비밀번호 입력('oracle')
- 롤 : SYSDBA 선택

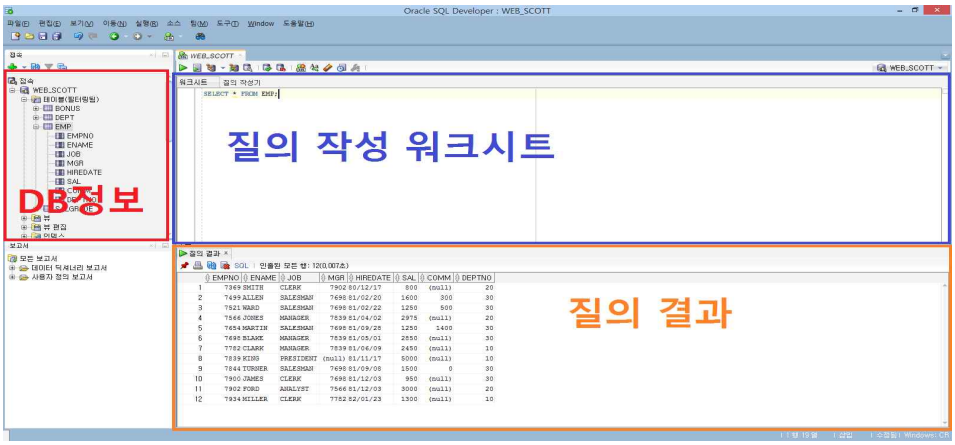
⑥ SQL Developer 접속 화면



☞ SQL Developer 실행파일에서 우클릭 - 보내기 - 바탕화면바로가기 클릭 하면 바탕화면에서 바로가기가 생성되어 쉽게 실행할 수 있다.

⑦ SQL Developer 작업 화면구성

- 작업 화면의 왼쪽 **:** DB 정보 확인
- 작업 화면 중앙 위쪽 : 질의 작성 워크시트
- 작업 화면 중앙 하단 : 질의 결과



1.2. 테스트용 Scott 계정 생성

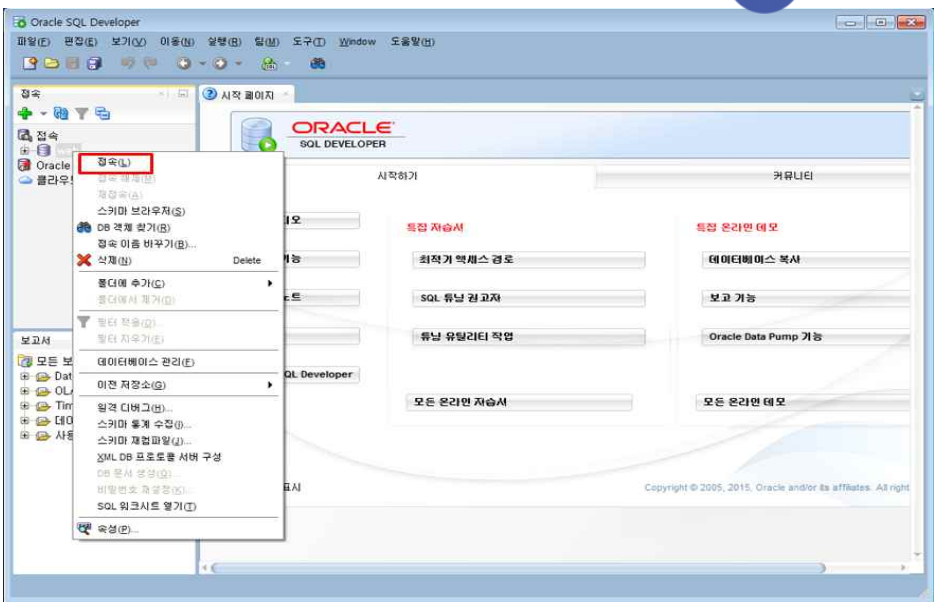
☞ 사용자 계정 및 테이블, 권한 등은 차후에 배우니 일단 따라 쓰기로 하자.

1.2.1 SQL Develope 사용

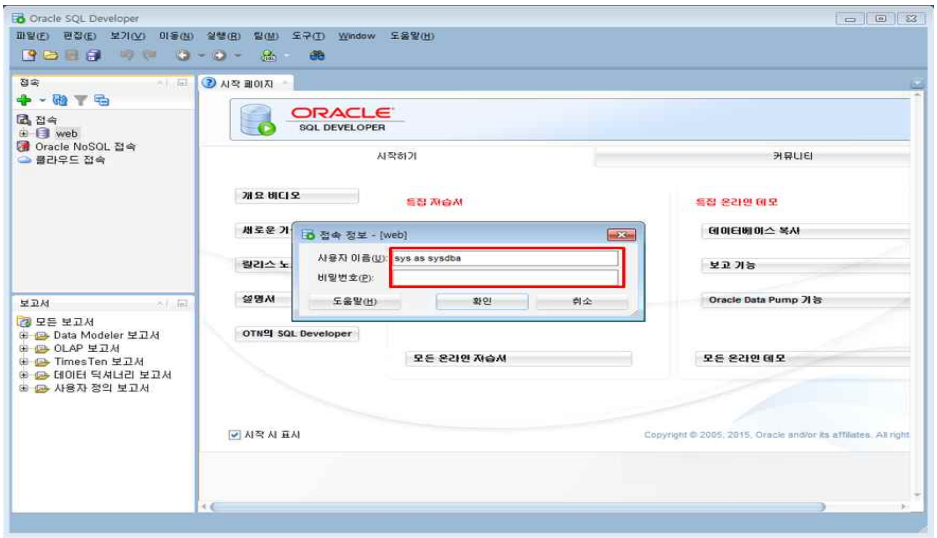
① sqldeveloper 실행

② sys as sysdba로 로그인

- sqldevelope실행화면 오른쪽 접속트리 하단 데이터베이스에서 우클릭하고 접속클릭



- 사용자 이름 : sys as sysdba db관리자 계정으로 접속 비밀번호 oracle



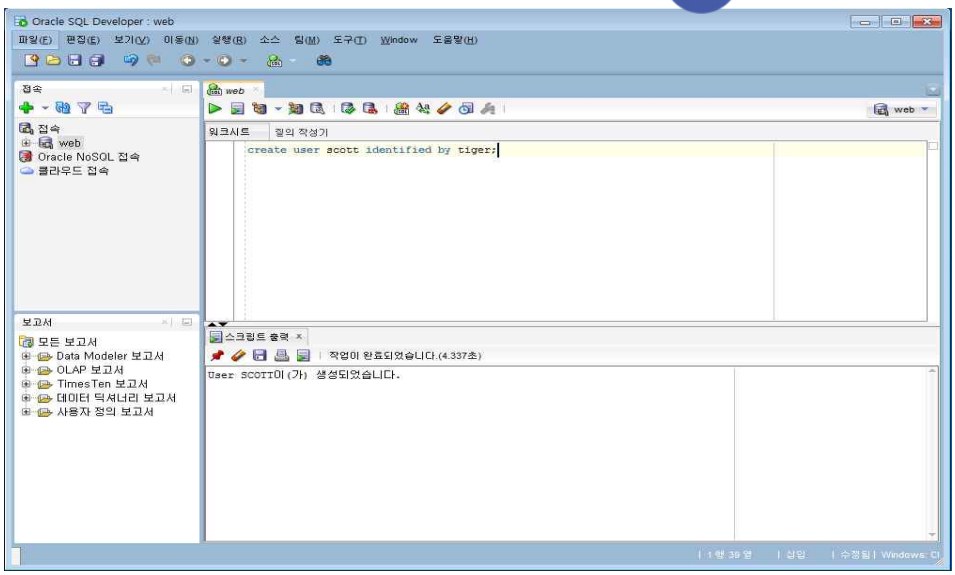
※ 반드시 sys의 DBA권한으로 접속해야지 계정을 생성 할 수 있다.

③ scott계정 생성

- 워크시트에 아래 명령어 입력 후 Ctlr+Enter 또는 상단 실행버튼 클릭

```
CREATE USER scott IDENTIFIED BY tiger;
```

실행결과



2. SQL Developer를 통한 사용자 계정 생성

2.1. Command Prompt(도스창)에서 SQLplus사용

① Run SQL Command Line 실행

② system 계정 로그인

- conn system/[데이터베이스패스워드]

③ scott 계정생성 및 계정비밀번호 설정

- create user scott;
- grant resource, connect to scott;
- alter user scott identified by tiger;

④ scott 계정 접속

- conn scott/tiger;

```
SQL*Plus: Release 11.2.0.2.0 Production on 월 12월 19 12:39:25 2016

Copyright (c) 1982, 2014, Oracle.  All rights reserved.

SQL> @C:\oraclexe\app\oracle\product\11.2.0\server\rdbms\admin\scott.sql
SQL> conn system/*****
Connected.
SQL> alter user scott identified by tiger;

User altered.

SQL> conn scott/tiger;
Connected.
SQL>
```

실습용 DB생성은 SQLDevelope와 동일한 명령어 수행

※ 실습용 DB는 인터넷에서 scott 스크립트라고 검색하면 scott.sql이라는 파일을 받을 수 있는데 그것을 활용하면 더욱 편리하다.

활용방법 : 파일을 편집으로 열어서 복사 붙여넣기, 혹은 sqlplus에서 @scott.sql입력

3. 주요 사용자계정

3.1. 주요 사용자계정 정의 및 종류

- 오라클에는 기본적으로 오라클이 제공하는 관리자용 계정 및 연습용 계정이 아래와 같이 존재한다. 오라클 연습을 위해 주로 scott계정을 사용한다.
- 관리자용 계정의 비밀번호는 오라클 설치시에 설정하니 설정한 후에는 기억해야한다.

계정명	설명
SYS	Super사용자 계정이며, 데이터베이스에서 발생하는 모든 문제들을 처리할 수 있는 권한을 가지고 있음.
SYSTEM	유지보수 및 관리할 때 사용하는 사용자계정,
ANONYMOUS	http를 통해 xml DB를 접근하기 위해 사용되는 유저
HR	샘플로 만들어 놓은 사용자 계정

본 교재에서는 sql developer와 scott 계정의 DB를 활용하여 실습을 진행한다.

실습 DB구조

EMP테이블

```
이름        널         유형
--------- -------- ------------
EMPNO     NOT NULL NUMBER
ENAME              VARCHAR2(10)
JOB                VARCHAR2(9)
MGR                NUMBER
HIREDATE           DATE
SAL                NUMBER
COMM               NUMBER
DEPTNO             NUMBER
```

DEPT테이블

```
이름     널 유형
------ - ------------
DEPTNO   NUMBER
DNAME    VARCHAR2(14)
LOC      VARCHAR2(13)
```

BONUS테이블

```
이름     널 유형
----- - ------------
ENAME    VARCHAR2(10)
JOB      VARCHAR2(9)
SAL      NUMBER
COMM     NUMBER
```

SALGRADE테이블

```
이름     널 유형
----- - ------
GRADE    NUMBER
LOSAL    NUMBER
HISAL    NUMBER
```

Chapter 5. 권한 설정하기

1. 계정 권한 설정

1.1. 테스트용 Scott 계정 권한 부여

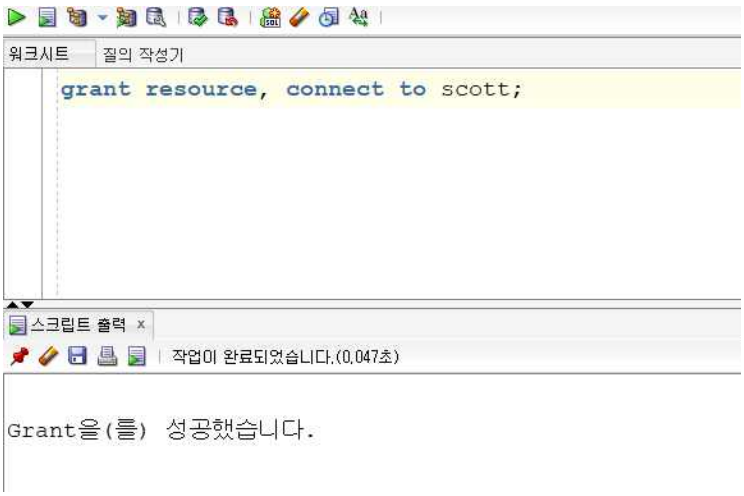
① scott계정에 권한부여

```
GRANT resource, connect TO scott;
```

*RESOURCE : 개체를 생성, 변경, 제거 할 수 있는 권한 (DDL,DML사용 가능)

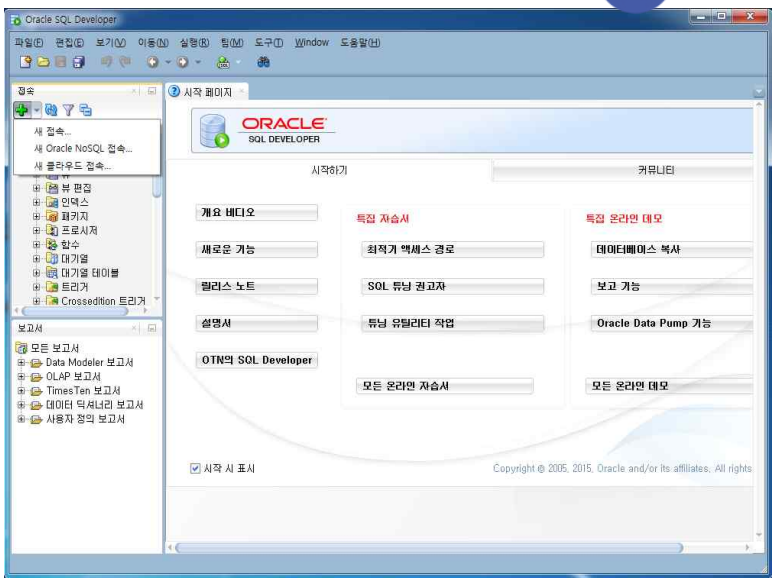
CONNECT : 데이터베이스에 연결할 수 있는 권한

실행결과

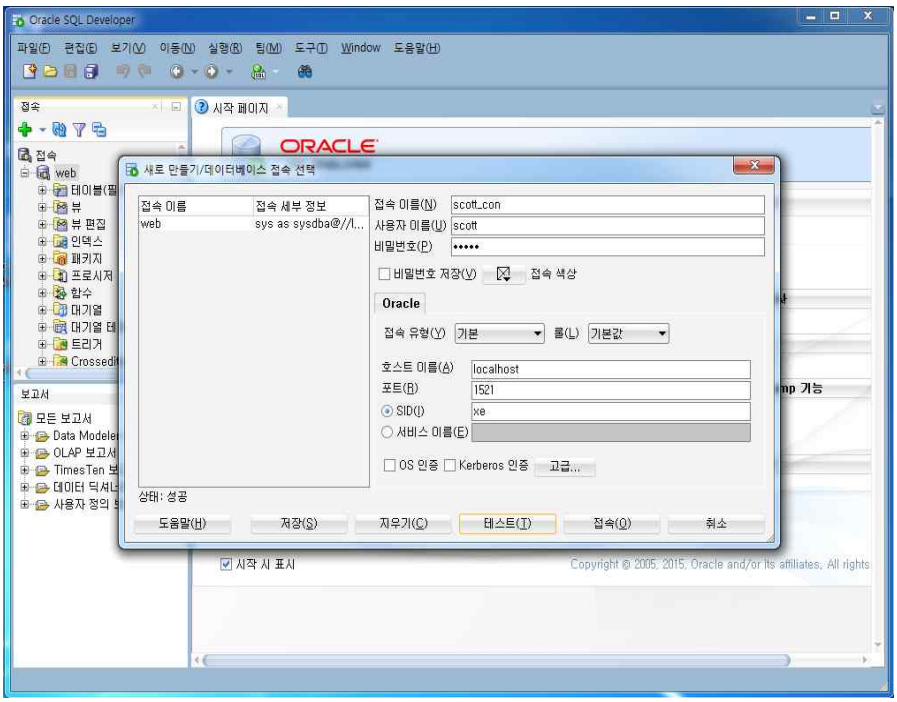


④ scott계정 로그인

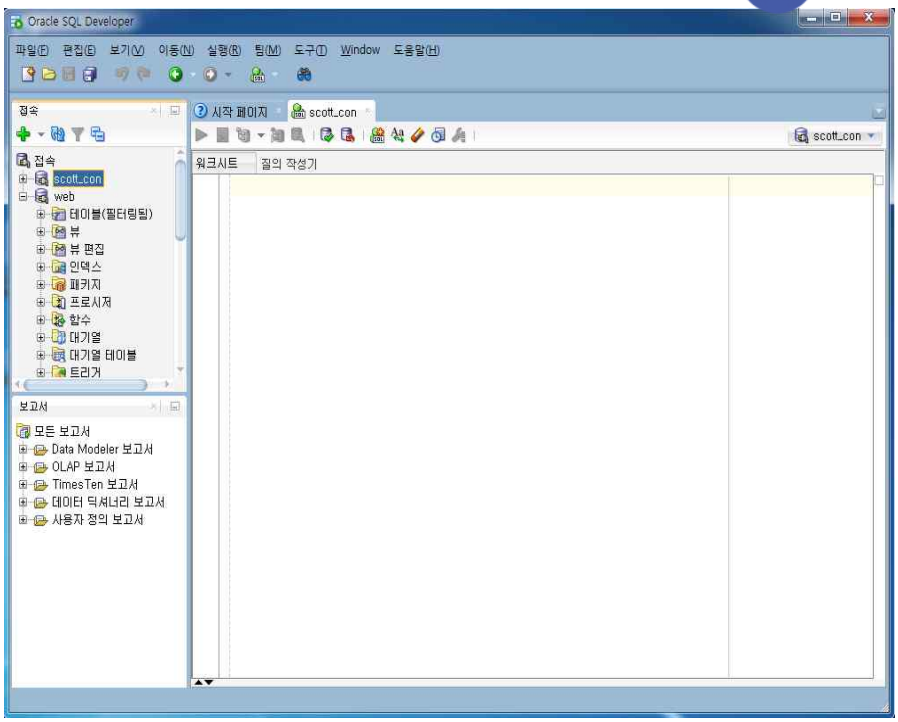
- 초기 화면 좌측 상단 '+'버튼 클릭 후 새접속 클릭



- 사용자이름 : scott 비밀번호 : tiger 입력 후 테스트- 접속



- 우측상단 테이블 정보에 접속이름으로 트리가 생기면 scott접속 성공



실습을 위한 계정 만들기를 완료하였고, 이제 실습용 DB를 구성해 보겠습니다. 아래 내용도 다음 단원에서 상세하게 배우니 일단은 따라 쓰도록 하자. 지금부터는 scott계정으로 로그인 후 진행 한다. 우측 상단에 새로 생긴 트리를 더블클릭하고 scott계정으로 로그인 하면 된다.

※ ;, ' ', 오타를 유의해서 작성해야 한다. 오타가 있으면 실행되지 않는다.

⑤ scott DB생성(실습 자료구성)

워크시트에 작성 후 F5 혹은 스크립트실행( ) 아이콘 클릭

- 테이블 초기화

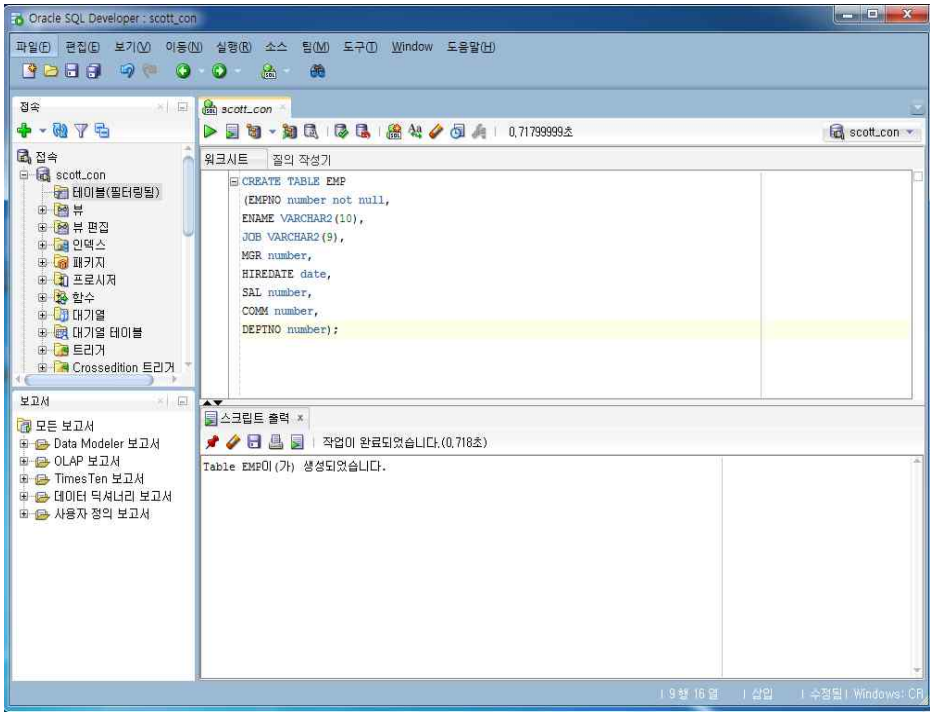
```
DROP TABLE emp;
DROP TABLE dept;
DROP TABLE bonus;
DROP TABLE salgrade;
```

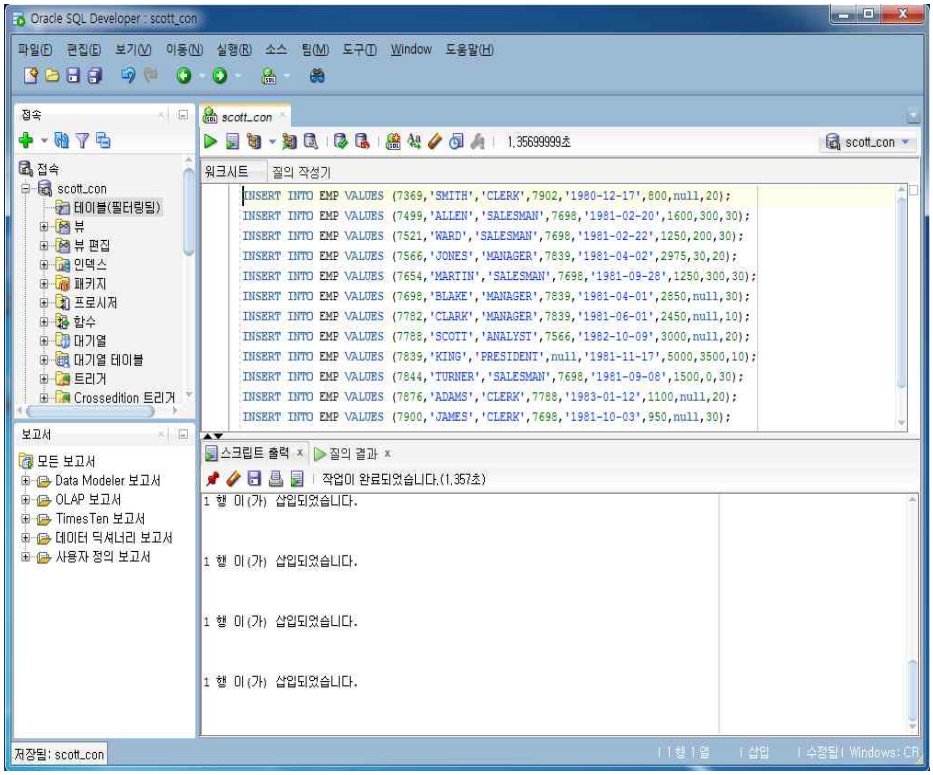
- EMP테이블 작성

```
CREATE TABLE EMP
(EMPNO number not null,
ENAME VARCHAR2(10),
JOB VARCHAR2(9),
MGR number,
HIREDATE date,
SAL number,
COMM number,
DEPTNO number);

INSERT INTO EMP VALUES (7369,'SMITH','CLERK',7902,'1980-12-17',800,null,20);
INSERT INTO EMP VALUES (7499,'ALLEN','SALESMAN',7698,'1981-02-20',1600,300,30);
INSERT INTO EMP VALUES (7521,'WARD','SALESMAN',7698,'1981-02-22',1250,200,30);
INSERT INTO EMP VALUES (7566,'JONES','MANAGER',7839,'1981-04-02',2975,30,20);
INSERT INTO EMP VALUES (7654,'MARTIN','SALESMAN',7698,'1981-09-28',1250,300,30);
INSERT INTO EMP VALUES (7698,'BLAKE','MANAGER',7839,'1981-04-01',2850,null,30);
INSERT INTO EMP VALUES (7782,'CLARK','MANAGER',7839,'1981-06-01',2450,null,10);
INSERT INTO EMP VALUES (7788,'SCOTT','ANALYST',7566,'1982-10-09',3000,null,20);
INSERT INTO EMP VALUES (7839,'KING','PRESIDENT',null,'1981-11-17',5000,3500,10);
INSERT INTO EMP VALUES (7844,'TURNER','SALESMAN',7698,'1981-09-08',1500,0,30);
INSERT INTO EMP VALUES (7876,'ADAMS','CLERK',7788,'1983-01-12',1100,null,20);
INSERT INTO EMP VALUES (7900,'JAMES','CLERK',7698,'1981-10-03',950,null,30);
INSERT INTO EMP VALUES (7902,'FORD','ANALYST',7566,'1981-10-3',3000,null,20);
INSERT INTO EMP VALUES (7934,'MILLER','CLERK',7782,'1982-01-23',1300,null,10);
```

실행결과


Oracle SQL Developer : scott_con
CREATE TABLE EMP
(EMPNO number not null,
ENAME VARCHAR2(10),
JOB VARCHAR2(9),
MGR number,
HIREDATE date,
SAL number,
COMM number,
DEPTNO number);
Table EMP이(가) 생성되었습니다.

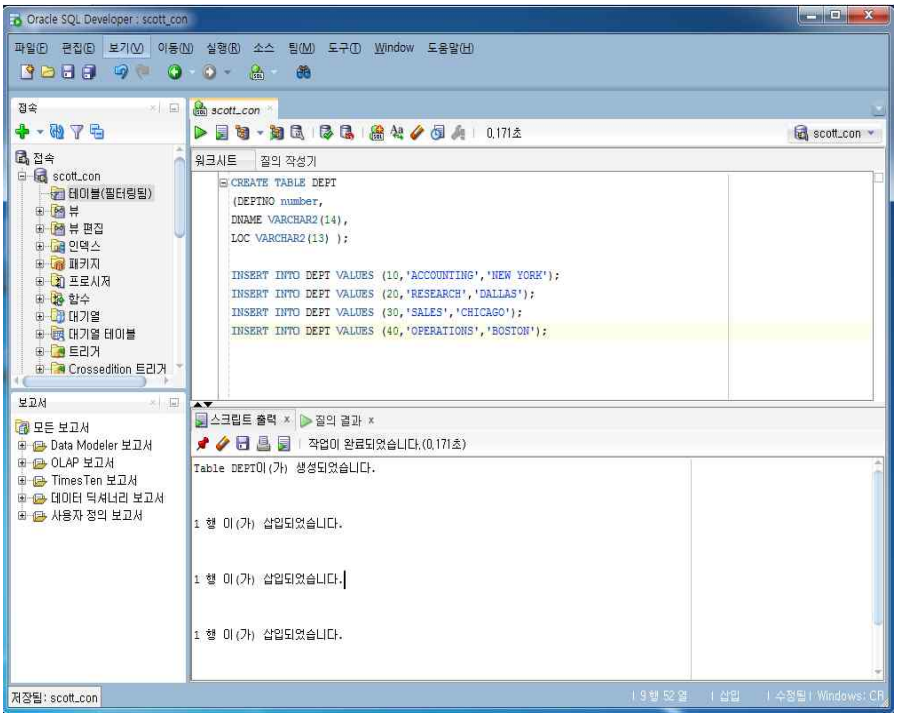

Oracle SQL Developer : scott_con
INSERT INTO EMP VALUES (7369,'SMITH','CLERK',7902,'1980-12-17',800,null,20);
INSERT INTO EMP VALUES (7499,'ALLEN','SALESMAN',7698,'1981-02-20',1600,300,30);
INSERT INTO EMP VALUES (7521,'WARD','SALESMAN',7698,'1981-02-22',1250,200,30);
INSERT INTO EMP VALUES (7566,'JONES','MANAGER',7839,'1981-04-02',2975,30,20);
INSERT INTO EMP VALUES (7654,'MARTIN','SALESMAN',7698,'1981-09-28',1250,300,30);
INSERT INTO EMP VALUES (7698,'BLAKE','MANAGER',7839,'1981-04-01',2850,null,30);
INSERT INTO EMP VALUES (7782,'CLARK','MANAGER',7839,'1981-06-01',2450,null,10);
INSERT INTO EMP VALUES (7788,'SCOTT','ANALYST',7566,'1982-10-09',3000,null,20);
INSERT INTO EMP VALUES (7839,'KING','PRESIDENT',null,'1981-11-17',5000,3500,10);
INSERT INTO EMP VALUES (7844,'TURNER','SALESMAN',7698,'1981-09-08',1500,0,30);
INSERT INTO EMP VALUES (7876,'ADAMS','CLERK',7788,'1983-01-12',1100,null,20);
INSERT INTO EMP VALUES (7900,'JAMES','CLERK',7698,'1981-10-03',950,null,30);
1 행 이(가) 삽입되었습니다.
1 행 이(가) 삽입되었습니다.
1 행 이(가) 삽입되었습니다.
1 행 이(가) 삽입되었습니다.

- DEPT테이블 작성

```
CREATE TABLE DEPT
(DEPTNO number,
DNAME VARCHAR2(14),
LOC VARCHAR2(13) );

INSERT INTO DEPT VALUES (10,'ACCOUNTING','NEW YORK');
INSERT INTO DEPT VALUES (20,'RESEARCH','DALLAS');
INSERT INTO DEPT VALUES (30,'SALES','CHICAGO');
INSERT INTO DEPT VALUES (40,'OPERATIONS','BOSTON');
```

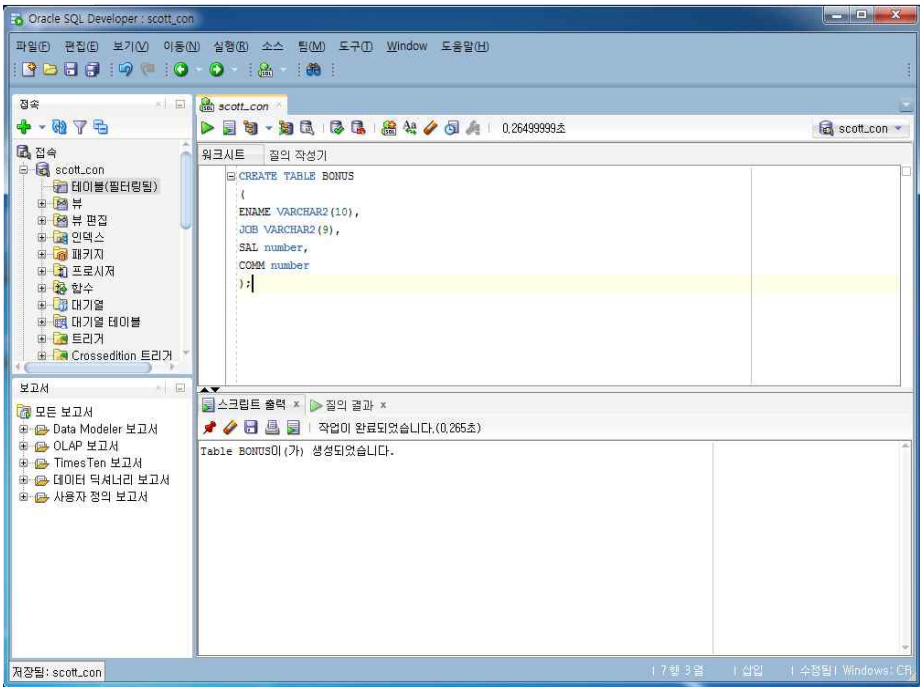
실행결과



- BONUS 테이블 작성

```
CREATE TABLE BONUS
(
ENAME VARCHAR2(10),
JOB VARCHAR2(9),
SAL number,
COMM number
);
```

실행결과

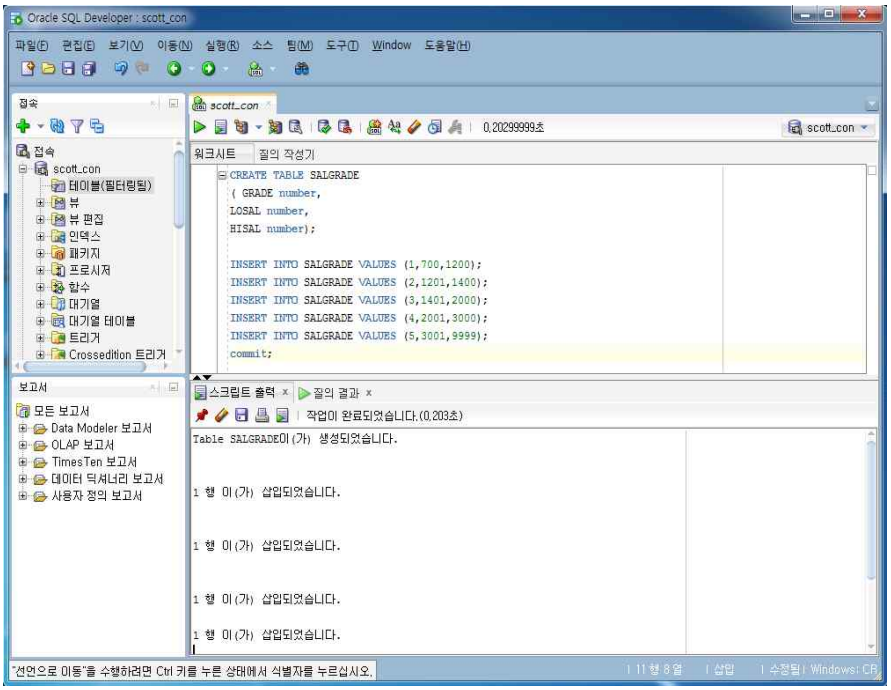


- SALGRADE 테이블 작성

```
CREATE TABLE SALGRADE
( GRADE number,
LOSAL number,
HISAL number);

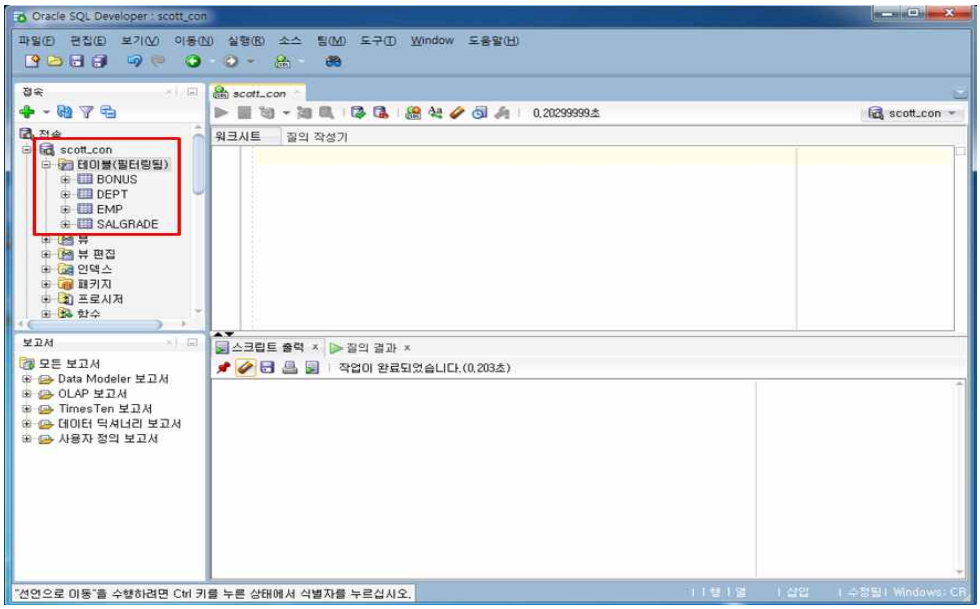
INSERT INTO SALGRADE VALUES (1,700,1200);
INSERT INTO SALGRADE VALUES (2,1201,1400);
INSERT INTO SALGRADE VALUES (3,1401,2000);
INSERT INTO SALGRADE VALUES (4,2001,3000);
INSERT INTO SALGRADE VALUES (5,3001,9999);
commit;
```

실행결과



☞ 마지막 commit 명령으로 DB에 해당 데이터를 저장한다.

완료 되면 오른쪽 상단 DB정보 트리에서 테이블을 클릭하면 테이블이 추가 된 것을 확인할 수 있다.



2. 오라클의 기본 데이터형

구 분	비 고
NUMBER	internal Number Format 최고 40자까지의 숫자를 저장 소수점이나 부호는 길이에 미포함
NUMBER(S)	S의 자리까지 최대 38자리까지 저장
NUMBER(S, D)	S는 전체 길이, D는 소수점 이하 자릿수 소수점은 자릿수 미포함
LOB	2GB까지의 가변길이 바이너리 데이터를 저장 이미지, 오디오, 파일,비디오 등 저장
DATE	날짜 및 시간 데이터 저장 BC 4712년 1월1월 ~ AD 4712년 12월 31일까지의 날짜
LONG	가변길이의 문자형 데이터 타입 최대크기 2GB
ROWID	Tree-piece Format으로 DB에 저장되지 않고 DBdate도 아님 테이블에서 행의 위치를 지정하는 논리적 주소값
BFILE	대용량의 바이너리 데이터를 파일 형태로 저장 최대크기 4GB
TIMESTAMP(i)	DATE형의 확장형으로 더 정밀한 값이 요구될 때 사용
CHAR(size)	문자열 데이터 저장 고정공간 할당(입력된 크기와 상관없지 공간할당) 사이즈를 byte|char 단위로 지정(기본단위는 byte) 예) CHAR(10 CHAR) : 영문/숫자/한글 상관없이 고정형 10자리 저장
VARCHAR2(size)	문자열 데이터 저장 가변공간 할당(입력된 크기만큼 공간할당) 사이즈를 byte|char 단위로 지정(기본단위는 byte) 예) VARCHAR2(10 CHAR) : 영문/숫자/한글 상관없이 최대크

	기 10자리까지 저장
NCHAR	문자열 데이터 저장 national character set의존
NVARCHAR2	문자열 데이터 저장 national character set의존

Chapter 6. DDL - TABLE 과 제약조건

테이블

데이터베이스의 기본 데이터 저장 단위인 테이블은 사용자가 접근 가능한 데이터를 보유하며 레코드(행)와 컬럼(열)으로 구성된다

시스템 내에서 독립적으로 사용되길 원하는 데이터 집합을 표현하며 데이터 집합 간의 관계를 표현할 수 있다. 테이블 간의 관계를 표현하는 데 외래 키(Foreign Key)를 사용한다.

DDL을 이용해 테이블을 생성(CREATE), 변경(ALTER), 제거(DROP)하는 등의 일을 수행할 수 있다

1. CREATE TABLE

CREATE TABLE을 이용해 테이블을 만들 때에는 각 컬럼들을 괄호 ( )로 묶어 지정

컬럼 뒤에 해당 컬럼의 데이터타입을 반드시 지정해야 한다

각 컬럼의 구분은 콤마(,)로 구분하며 CREATE TABLE 구문의 마지막은 세미콜론(;)으로 끝난다

테이블 내의 컬럼들은 같은 이름을 가질 수 없다

1. CREATE TABLE 문법

```
CREATE TABLE table_name (
 column1 자료형1
, column2 자료형2
, ...
);
```

```
CREATE TABLE test (
        empno NUMBER(4),
        ename VARCHAR2(30),
        sal NUMBER(7, 2)
);
```

2. 사용자가 소유하는 모든 테이블 조회

```
SQL> SELECT table_name FROM USER_TABLES;
```

```
TABLE_NAME
----------
DEPT
```

EMP
BONUS
SALGRADE
TEST

1.1. 데이터 타입

1.1.1. CHAR

고정 길이 문자 데이터
VARCHAR2와 동일한 형태의 데이터를 저장할 수 있다
입력된 데이터의 크기와 무관하게 고정된 길이만큼 저장 공간을 차지한다
저장 공간의 크기를 명시해야 하며, 최소 크기는 1
예) char(10)

1.1.2. VARCHAR2

가변 길이 문자 데이터
실제 입력된 데이터의 크기만큼 저장 공간을 차지한다
최대 크기를 명시해야 하며, 최소 크기는 1
예) varchar2(30)

1.1.3. NUMBER

숫자를 나타내기 위한 데이터 타입
음수, 0, 양수를 저장할 수 있다
연산이 필요한 컬럼에 NUMBER타입을 지정한다
가변길이 데이터 타입이다
소수점이 지정되지 않았는데 입력되거나, 지정된 소수점 자릿수 이상 입력되면 반올림되어 저장

3. NUMBER 타입의 3가지 방법

NUMBER	최고 38자리까지의 숫자 저장 가능
NUMBER(w)	w자리까지의 수치로 최대 38자리까지 가능 (유효숫자가 38자리)
NUMBER(w, d)	w는 전체 자릿수, d는 소수점 이하 자릿수를 지정 w의 길이는 d의 길이를 포함

1.1.4. DATE

날짜와 시간을 저장하는 날짜형 데이터 타입

BC 4712년 1월 1일 ~ AD 4712년 12월 31일까지의 날짜를 저장할 수 있다. 날짜와 시간의 연산이 빈번한 경우 사용한다. 세기, 년도, 월, 일, 시간, 분, 초 데이터를 포함한다

SYSDATE는 현재 날짜와 시간을 제공한다. (DATE타입)

1.1.5. TIMESTAMP

DATE형의 확장된 형태

밀리초 단위까지 표현 가능한 데이터타입

1.1.6. LONG

가변 길이 문자형 데이터 타입

최대 크기는 2GB

테이블에 한 개의 컬럼만 생성 가능

SELECT의 결과로 데이터가 직접 조회된다

1.1.7. CLOB

가변 길이 문자형 데이터 타입

최대 크기는 4GB

테이블에 여러 개의 컬럼을 생성할 수 있다

SELECT의 결과로 데이터의 위치가 조회된다

1.1.8. RAW, LONG RAW

이미지나 사운드를 저장할 수 있는 데이터 타입

이진 데이터를 저장하며 16진수 형태로 표현된다

VARCHAR2 데이터타입과 유사하지만 문자변환을 수행할 수 없다

1.1.9. BFILE

대용량 이진 데이터를 파일 형태로 저장하는 데이터 타입

최대 크기는 4GB

1.2. 명명 규칙

테이블명과 컬럼명을 사용자가 부여할 때 지켜야하는 규칙이다

4. 문자로 시작
5. 1~30자까지 가능

6. A~Z의 대소문자, 0~9의 숫자, 특수기호(_, $, #) 만 가능
7. 오라클의 예약어나 다른 객체명과 중복 불가
8. 공백을 허용하지 않는다

1.3. 제약 조건

테이블에 제약사항을 걸어 부적절한 데이터가 입력되는 것을 막기 위한 것이다

데이터 무결성을 유지하기 위해 사용자가 지정하는 성질이다

CREATE TABLE 구문에서 지정할 수 있으며, 테이블 생성 후에 ALTER TABLE table_name ADD CONSTRAINTS 명령어를 이용하여 추가하는 것도 가능하다

테이블생성시 제약조건 생성하기

1. 컬럼레벨에서 작성

```
--컬럼명 없이 제약조건 생성
--제약사항의 이름을 지정하지 않은 경우 SYS_xxxxxxx 와 같은 이름의 제약사항으로 생성된다

CREATE TABLE emp01(
        empno NUMBER PRIMARY KEY
      , ename VARCHAR2(20) NOT NULL
      , job VARCHAR2(9) UNIQUE
      , deptno NUMBER(2) REFERENCES dept(deptno)
);

--USER_CONSTRAINTS 테이블에서 확인
SELECT CONSTRAINT_NAME, CONSTRAINT_TYPE, TABLE_NAME, R_CONSTRAINT_NAME

FROM user_constraints

WHERE table_name='EMP01';
```

결과

	CONSTRAINT_NAME	CONSTRAINT_TYPE	TABLE_NAME	R_CONSTRAINT_NAME
1	SYS_C007047	C	EMP01	(null)
2	SYS_C007048	P	EMP01	(null)
3	SYS_C007049	U	EMP01	(null)
4	SYS_C007050	R	EMP01	SYS_C007046

```
--컬럼명 추가해서 제약조건 생성

--자료형 다음에 [CONSTRAINTS 제약조건명 제약조건]의 순으로 추가한다.


CREATE TABLE emp02(
        empno NUMBER CONSTRAINTS EMP02_EMPNO_PK PRIMARY KEY
      , ename VARCHAR2(20) CONSTRAINTS EMP02_ENAME_NN NOT NULL
      , job VARCHAR2(9) CONSTRAINTS EMP02_JOB_UK UNIQUE
      , deptno NUMBER(2) CONSTRAINTS EMP02_DEPTNO_FK REFERENCES dept(deptno)
);


--USER_CONSTRAINTS 테이블에서 확인
SELECT CONSTRAINT_NAME, CONSTRAINT_TYPE, TABLE_NAME, R_CONSTRAINT_NAME

FROM user_constraints

WHERE table_name='EMP02';
```

결과

	CONSTRAINT_NAME	CONSTRAINT_TYPE	TABLE_NAME	R_CONSTRAINT_NAME
1	EMP02_ENAME_NN	C	EMP02	(null)
2	EMP02_EMPNO_PK	P	EMP02	(null)
3	EMP02_JOB_UK	U	EMP02	(null)
4	EMP02_DEPTNO_FK	R	EMP02	SYS_C007046

2. 테이블레벨에서 작성

NOT NULL 제약조건은 테이블레벨에서 작성할 수 없다. 테이블생성시 컬럼레벨에서 작성하거나, 테이블 생성 이후 ALTER TABLE table_name MODIFY문을 통해서 추가해야 한다.

```
--컬럼명 없이 제약조건 생성
--제약사항의 이름을 지정하지 않은 경우 SYS_xxxxxxx 와 같은 이름의 제약사항으로 생성된다


CREATE TABLE emp03(
        empno NUMBER
```

```
        , ename VARCHAR2(20) NOT NULL
        , job VARCHAR2(9)
        , deptno NUMBER(2)
        , PRIMARY KEY(empno)
        , UNIQUE(job)
        , FOREIGN KEY(deptno) REFERENCES dept(deptno)
);

--USER_CONSTRAINTS 테이블에서 확인
SELECT CONSTRAINT_NAME, CONSTRAINT_TYPE, TABLE_NAME, R_CONSTRAINT_NAME

FROM user_constraints

WHERE table_name='EMP03';
```

결과

	CONSTRAINT_NAME	CONSTRAINT_TYPE	TABLE_NAME	R_CONSTRAINT_NAME
1	SYS_C007055	C	EMP03	(null)
2	SYS_C007056	P	EMP03	(null)
3	SYS_C007057	U	EMP03	(null)
4	SYS_C007058	R	EMP03	SYS_C007046

```
--제약조건명 추가해서 제약조건 생성

CREATE TABLE emp04(
        empno NUMBER
        , ename VARCHAR2(20) CONSTRAINTS EMP02_ENAME_NN NOT NULL
        , job VARCHAR2(9)
        , deptno NUMBER(2)
        , CONSTRAINTS EMP02_EMPNO_PK PRIMARY KEY
        , CONSTRAINTS EMP02_JOB_UK UNIQUE
        , CONSTRAINTS EMP02_DEPTNO_FK REFERENCES dept(deptno)
);

--USER_CONSTRAINTS 테이블에서 확인
SELECT CONSTRAINT_NAME, CONSTRAINT_TYPE, TABLE_NAME, R_CONSTRAINT_NAME
```

```
FROM user_constraints

WHERE table_name='EMP04';
```

결과

	CONSTRAINT_NAME	CONSTRAINT_TYPE	TABLE_NAME	R_CONSTRAINT_NAME
1	EMP04_ENAME_NN	C	EMP04	(null)
2	EMP04_EMPNO_PK	P	EMP04	(null)
3	EMP04_JOB_UK	U	EMP04	(null)
4	EMP04_DEPTNO_FK	R	EMP04	SYS_C007046

1.3.1. NOT NULL

해당 컬럼에 NULL값이 입력될 수 없도록 한다

```
-- 제약사항의 이름을 지정하지 않고 NOT NULL 제약을 적용
CREATE TABLE test (
        data varchar2(20) NOT NULL
);
```

```
-- 제약사항의 이름을 test_nn_data로 지정하고 NOT NULL 제약을 적용
CREATE TABLE test (
        DATA VARCHAR2(20) CONSTRAINT test_nn_data NOT NULL
);
```

* 제약사항의 이름을 지정하지 않은 경우 SYS_xxxxxxx 와 같은 이름의 제약사항으로 생성된다

1.3.2. UNIQUE

데이터의 유일성을 보장하는 제약 조건

해당 컬럼에 중복 데이터가 존재할 수 없게 되며 자동 인덱스가 생성된다

```
CREATE TABLE test (
        data1 varchar2(20) UNIQUE,
        data2 varchar2(20)
);
```

```
-- 테이블 생성 후에 data2 컬럼에 UNIQUE 제약사항 변경
ALTER TABLE TEST ADD CONSTRAINT test_uk_data2 UNIQUE (data2);
```

1.3.3. CHECK

컬럼의 값을 특정 범위로 제한

```
CREATE TABLE test (
        data1 NUMBER,
        data2 NUMBER CHECK (data2 >=1 AND data2 <= 99)
);
```

```
-- 테이블 생성 후에 data2 컬럼에 UNIQUE 제약사항 변경
ALTER TABLE TEST ADD CONSTRAINT test_ck_data1 CHECK data1 IN (1000, 2000, 3000, 4000);
```

1.3.4. PRIMARY KEY

기본키는 UNIQUE와 NOT NULL 제약사항을 모두 포함한다

기본키는 행을 대표하는 컬럼으로 외래키들이 참조할 수 있는 키값의 자격을 가지고 있다

자동으로 인덱스를 생성하며 기본키 제약 조건의 이름과 같은 이름을 가진다

```
CREATE TABLE TEST (
        no NUMBER CONSTRAINT test_pk_no PRIMARY KEY,
        data VARCHAR2(30)
);
```

```
-- 테이블 생성 후에 data 컬럼에 기본키 제약사항을 생성할 경우
ALTER TABLE TEST ADD CONSTRAINT test_uk_data PRIMARY KEY (no);
```

1.3.5. FOREIGN KEY

기본키를 참조하는 컬럼 또는 컬럼의 집합을 뜻한다

참조하고 있는 기본키의 데이터타입과 일치해야 하며 외래키에 참조되고 있는 기본키는 삭제할 수 없는 제약이 걸린다

```
-- 기본키를 가지는 test테이블 생성
CREATE TABLE TEST (no NUMBER CONSTRAINT test_pk_no PRIMARY KEY, data VARCHAR2(30));
```

```
-- test2 테이블 생성
CREATE TABLE TEST2 (no NUMBER );
```

```
-- test 테이블의 기본키 no 컬럼을 참조하는 외래키 제약사항 설정
ALTER TABLE TEST2 ADD CONSTRAINT test2_fk_no   FOREIGN KEY (no) REFERENCES test(no);
```

ON DELETE [CASCADE/SET NULL] 연산자를 통해 기본키가 삭제될 경우 같이 삭제된다.

외래키로 지정된 기본키를 삭제하는 경우, 외래키의 참조무결성에 의해 기본키를 삭제할 수 없다. 그 번거로움을 덜기 위해서 외래키를 설정할 때, 기본키 삭제시 따라 삭제 또는 NULLABLE로 제약조건을 변경할 수 있다.

- ON DELETE CASCADE : 부모테이블기본키(parentTbl의 name) 삭제시, 자식테이블의 name컬럼 제약조건도 따라 삭제.

```
CREATE TABLE test1 (
no NUMBER(4) PRIMARY KEY,
name VARCHAR2(10) CONSTRAINT test4_fk REFERENCES parentTbl(name) ON DELETE CASCADE

) ;
```

- ON DELETE SET NULL : 부모테이블기본키(parentTbl의 name) 삭제시, 자식테이블의 name컬럼 제약조건은 NULLABLE로 변경.

```
CREATE TABLE test1 (
no NUMBER(4) PRIMARY KEY,
name VARCHAR2(10) CONSTRAINT test4_fk REFERENCES parentTbl(name) ON DELETE SET NULL

) ;
```

1.3.6. 제약사항 확인

9. 제약사항 정보를 확인 할수 있는 테이블(데이터사전)

   USER_CONS_COLUMNS : 컬럼에 할당된 제약 조건 조회

   USER_CONSTRAINTS : 유저가 소유한 모든 제약 조건 조회

10. 사용자의 모든 제약사항 확인

```
SQL> SELECT * FROM USER_CONSTRAINTS;
```

11. 사용자의 모든 제약사항을 보기 편하게 출력하는 SQL

```
SQL> SELECT
        SUBSTR(A.COLUMN_NAME,1,15) COLUMN_NAME,
        DECODE(B.CONSTRAINT_TYPE,
                'P','PRIMARY KEY',
```

```
                    'U','UNIQUE KEY',
                    'C','CHECK OR NOT NULL',
                    'R','FOREIGN KEY'
            ) CONSTRAINT_TYPE,
            A.CONSTRAINT_NAME
FROM USER_CONS_COLUMNS A, USER_CONSTRAINTS B
WHERE A.TABLE_NAME = UPPER('&table_name')
            AND A.TABLE_NAME = B.TABLE_NAME
            AND A.CONSTRAINT_NAME = B.CONSTRAINT_NAME
ORDER BY 1;
```

2. ALTER TABLE

테이블 구조를 변경하기 위해서 사용하는 DDL

컬럼과 제약조건에 대하여, ADD(추가)/RENAME(이름변경)/MODIFY(수정)/DROP(삭제)를 수행할 수 있다.

2.1. 컬럼 추가/이름변경/수정/삭제

ALTER TABLE ~ ADD 컬럼

기존 테이블에 새로운 컬럼을 추가하는 명령어

새로운 컬럼은 테이블 맨 마지막에 추가된다

원하는 위치에 컬럼을 지정해서 넣을 수 없다

데이터가 존재하는 로우에는 컬럼이 추가되지만 컬럼 값은 NULL값으로 입력된다

test 테이블에 job 컬럼 추가하기

```
SQL> DESC test;
이름                    널 유형
------                  ------------
EMPNO                   NUMBER(4)
ENAME                   VARCHAR2(30)
SAL                     NUMBER(7,2)
SQL> ALTER TABLE test ADD (job VARCHAR(9));
Table TEST이(가) 변경되었습니다.
SQL> DESC test;
이름                    널 유형
```

```
------          ------------
EMPNO           NUMBER(4)
ENAME           VARCHAR2(30)
SAL             NUMBER(7,2)
JOB             VARCHAR2(9)
```

ALTER TABLE ~ RENAME 컬럼

컬럼명을 바꿀 때 사용하는 명령어

1. test 테이블의 ename 컬럼명을 empname으로 변경

```
SQL> DESC test;
이름            널 유형
------          ------------
EMPNO           NUMBER(4)
ENAME           VARCHAR2(30)
SQL> ALTER TABLE test RENAME COLUMN ename to empname;
Table TEST이(가) 변경되었습니다.
SQL> DESC test;
이름            널 유형
------          ------------
EMPNO           NUMBER(4)
EMPNAME         VARCHAR2(30)
```

ALTER TABLE ~ MODIFY 컬럼

테이블에 이미 존재하는 컬럼을 수정하는 명령어

해당 컬럼에 데이터 유무에 따라 동작이 다르다

데이터 유무에 따른 컬럼 변경

	자료가 없는 경우	자료가 있는 경우
데이터 타입 변경	변경 가능	변경 불가 (조건부)
컬럼의 크기 변경	변경 가능	작은 크기로는 불가

* 자료가 있는 경우 데이터타입이 가능한 경우는 CHAR와 VARCHAR2 뿐이다. 이 때, 데이터 손실이 없도록 기록된 최대값을 고려해서 변경하여야 한다.

test 테이블의 job 컬럼의 크기 변경

```
SQL> ALTER TABLE test MODIFY (job VARCHAR(30));
Table TEST이(가) 변경되었습니다.
SQL> DESC test;
 이름            널 유형
 ------          ------------
 EMPNO           NUMBER(4)
 ENAME           VARCHAR2(30)
 SAL             NUMBER(7,2)
 JOB             VARCHAR2(30)
```

컬럼의 DEFAULT 값을 변경하는 경우, 변경시점부터 기본값이 적용된다. 기존데이터들은 영향을 받지 않는다.

```
SQL> ALTER TABLE EMP MODIFY COMM NUMBER DEFAULT 0;
Table EMP이(가) 변경되었습니다.
SQL> INSERT INTO EMP_COPY (EMPNO, ENAME, JOB, DEPTNO) VALUES
(9999,'JANE','SALESMAN',30);
1행이 추가되었습니다.
SQL> SELECT * FROM EMP_COPY;
```

결과

	EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
11	7876	ADAMS	CLERK	7788	83/01/12	1100	(null)	20
12	7900	JAMES	CLERK	7698	81/10/03	950	(null)	30
13	7902	FORD	ANALYST	7566	81/10/03	3000	(null)	20
14	7934	MILLER	CLERK	7782	82/01/23	1300	(null)	10
15	1001	ALICE	CLERK	1003	16/09/08	800	(null)	30
16	1002	MORRIS	CLERK	1003	16/09/08	800	(null)	30
17	1003	MATT	SALEMAN	(null)	16/01/01	1500	100	30
18	1010	BOB	(null)	(null)	(null)	(null)	(null)	(null)
19	1011	EDWARD	MANAGER	(null)	(null)	(null)	(null)	(null)
20	1006	JORDAN	MANAGER	7782	82/02/05	5000	(null)	10
21	1234	JONATHAN	(null)	(null)	(null)	(null)	(null)	30
22	9999	JANE	SALESMAN	(null)	(null)	(null)	0	30

위와 같이 데이터 추가시 COMM컬럼값을 주지 않았는데, 0이라는 기본값이 지정되었다. 이미 존재하던 다른 행의 COMM컬럼값에는 변함이 없다.

ALTER TABLE ~ DROP 컬럼

테이블에 존재하는 컬럼을 삭제하는 명령어

test 테이블의 sal 컬럼 삭제

```
SQL> ALTER TABLE test DROP COLUMN sal;
Table TEST이(가) 변경되었습니다.
SQL> DESC test;
이름            널 유형
------          ------------
EMPNO           NUMBER(4)
ENAME           VARCHAR2(30)
JOB             VARCHAR2(30)
```

2.2. 제약조건 추가/이름변경/수정/삭제

ALTER TABLE ~ ADD 제약조건

기존 테이블에 새로운 제약조건을 추가하는 명령어

```
--형식
ALTER TABLE table_name ADD [CONSTRAINT constraint_name] constraint_type (column_name);
```

제약조건이름없이 제약조건을 생성한다면, CONSTAINT키워드는 생략가능하다. 제약조건의 이름을 지정하지 않은 경우 SYS_xxxxxxx 와 같은 이름의 제약사항으로 생성된다

test 테이블에 제약조건 추가하기

쿼리문

```
ALTER TABLE EMP ADD CONSTRAINTS EMP_EMPNO_PK PRIMARY KEY(EMPNO);
```

```
Table EMP이(가) 변경되었습니다.
```

```
SELECT CONSTRAINT_NAME, CONSTRAINT_TYPE, TABLE_NAME, R_CONSTRAINT_NAME
FROM USER_CONSTRAINTS
WHERE TABLE_NAME='EMP'
```

결 과

	CONSTRAINT_NAME	CONSTRAINT_TYPE	TABLE_NAME	R_CONSTRAINT_NAME
1	SYS_C006997	C	EMP	(null)
2	EMP_EMPNO_PK	P	EMP	(null)

ALTER TABLE ~ RENAME 제약조건

제약조건명을 바꿀 때 사용하는 명령어

쿼리문

```
ALTER TABLE EMP RENAME CONSTRAINTS EMP_EMPNO_PK TO EMP_EMPNO_PK_NEW;
```

```
Table EMP이(가) 변경되었습니다.
```

```
SELECT CONSTRAINT_NAME, CONSTRAINT_TYPE, TABLE_NAME, R_CONSTRAINT_NAME
FROM USER_CONSTRAINTS
WHERE TABLE_NAME='EMP'
```

결 과

	CONSTRAINT_NAME	CONSTRAINT_TYPE	TABLE_NAME	R_CONSTRAINT_NAME
1	SYS_C006997	C	EMP	(null)
2	EMP_EMPNO_PK_NEW	P	EMP	(null)

ALTER TABLE ~ MODIFY 제약조건

NOT NULL을 제외한 제약조건은 수정이 불가하다. 삭제(DROP)후 다시 생성(ADD)해야 한다.

NOT NULL 제약조건은 ADD가 아닌 MODIFY명령어를 사용해서 추가해야 한다. NULLABLE(기본값)에서 NOT NULL로 변경하는 것으로 이해해야 한다.

쿼리문

```
ALTER TABLE EMP MODIFY ENAME CONSTRAINTS EMP_ENAME_NN NOT NULL;
```

```
Table EMP이(가) 변경되었습니다.
```

```
SELECT CONSTRAINT_NAME, CONSTRAINT_TYPE, TABLE_NAME, R_CONSTRAINT_NAME
FROM USER_CONSTRAINTS
WHERE TABLE_NAME='EMP'
```

결 과

	CONSTRAINT_NAME	CONSTRAINT_TYPE	TABLE_NAME	R_CONSTRAINT_NAME
1	SYS_C006997	C	EMP	(null)
2	EMP_EMPNO_PK	P	EMP	(null)
3	EMP_ENAME_NN	C	EMP	(null)

ALTER TABLE ~ DROP 제약조건

테이블에 존재하는 제약조건을 삭제하는 명령어

```
--형식
ALTER TABLE table_name DROP [CONSTRAINT constraint_name];
```

기본적으로 제약조건명을 제시해서 삭제해야 한다. 제약조건명은 USER_CONSTRAINTS테이블에서 조회가능하므로, 정확한 제약조건명을 조회후 실행하도록 하자. 하지만 아래와 같이도 삭제가능하다.

```
ALTER TABLE table_name DROP PRIMARY KEY;
```

쿼리문

```
ALTER TABLE EMP DROP CONSTRAINTS EMP_ENAME_NN;
```

```
Table EMP이(가) 변경되었습니다.
```

```
SELECT CONSTRAINT_NAME, CONSTRAINT_TYPE, TABLE_NAME, R_CONSTRAINT_NAME
FROM USER_CONSTRAINTS
WHERE TABLE_NAME='EMP'
```

결 과

	CONSTRAINT_NAME	CONSTRAINT_TYPE	TABLE_NAME	R_CONSTRAINT_NAME
1	SYS_C006997	C	EMP	(null)
2	EMP_EMPNO_PK	P	EMP	(null)

3.3 ALTER TABLE ~ SET UNUSED

컬럼을 삭제하지 않지만 논리적으로 컬럼의 사용을 제한하는 옵션(비활성)

컬럼을 삭제하려는 테이블에 여러 사용자가 접근할 수 있기 때문에 무조건 삭제해버리는 것은 위험할 수 있다. 컬럼을 삭제할 경우 컬럼에 해당하는 데이터도 같이 삭제 해야하므로 데이터가 많을 경우 오랜 시간이 소요될 수 있다. 또한 삭제 작업을 할 경우 데이터 무결성을 위하여 다른 사용자의 접근을 막을 수 있다(LOCK 발생). 우선 SET UNUSED를 이용하여 비활성시킨 후에 작업량이 적은 시간대에 DROP을 하도록 한다.

단, ALTER TABLE ~ SET UNUSED 구문을 이용해 컬럼을 논리적으로 사용할 수 없게 만들 경우에도 DROP과 마찬가지로 데이터를 복구할 수 없다

test 테이블의 job 컬럼을 비활성

```
SQL> DESC test;
이름            널 유형
------          ----------
EMPNO           NUMBER(4)
ENAME           VARCHAR2(30)
JOB             VARCHAR2(30)
```

```
SQL> ALTER TABLE test SET UNUSED(job);
```

```
Table TEST이(가) 변경되었습니다.
```

```
SQL> DESC test;
이름            널 유형
------          ------------
EMPNO           NUMBER(4)
ENAME           VARCHAR2(30)
```

SET UNUSED 작업을 처리한 job 컬럼이 보이지 않는 걸 확인할 수 있다

3.4 ALTER TABLE ~ RENAME TO 새테이블명

테이블명을 바꿀 때 사용하는 명령어

test 테이블명을 test_ddl로 변경

```
SQL> SELECT table_name FROM tabs WHERE table_name LIKE '%TEST%';
TABLE_NAME
---------------
TEST
SQL> ALTER TABLE test RENAME TO test_ddl;
Table TEST이(가) 변경되었습니다.
SQL> SELECT table_name FROM tabs WHERE table_name LIKE '%TEST%';
TABLE_NAME
---------------
TEST_DDL
```

3. TRUNCATE TABLE

사용하던 테이블의 모든 행을 제거하기 위한 명령어

DROP TABLE 명령을 수행하면 테이블 존재 자체가 없어지는 것과는 달리 TRUNCATE TABLE은 테이블 기본 구조를 유지하면서 모든 데이터를 삭제한다

12. test_ddl 테이블에 TRUNCATE TABLE 명령 수행

```
SQL> TRUNCATE TABLE test_ddl;
Table TEST_DDL이(가) 잘렸습니다.
```

5. DROP TABLE

테이블을 삭제하는 명령어

테이블에 존재하던 데이터는 삭제되며 테이블도 삭제된다

삭제 후 되돌릴 수 없으니 주의하여야 한다

test_ddl 테이블에 TRUNCAT TABLE 명령 수행

```
SQL> DROP TABLE test_ddl;
Table TEST_DDL이(가) 삭제되었습니다.
```

5.1 DELETE, DROP, TRUNCATE의 차이

DELETE, DROP TABLE, TRUNCATE TABLE을 수행하면 모두 테이블의 데이터가 삭제되는 상황이지만 차이점이 있다

	종류	ROLLBACK 가능여부	비고
DELETE	DML	가능	• 데이터가 지워지지만 테이블의 용량이 줄어들지 않는다. • TRIGGER가 걸려있다면 각 행이 삭제될 때마다 실행된다. • 이전에 할당되었던 영역은 삭제되어 빈 TABLE이나 CLUSTER에 그대로 남아 있게 된다.
TRUNCATE TABLE	DDL	불가능	• 데이터가 지워지며 테이블의 용량이 줄어든다. • TABLE과 관련된 구조(CONSTRAINT, TRIGGER 등)과 권한에 영향을 주지 않는다. • TABLE에 걸려 있는 TRIGGER는 실행되지 않는다.
DROP TABLE	DDL	불가능	• 데이터뿐만 아니라 테이블 자체가 삭제된다. • 모든 관련된 INDEX, CONSTRAINT, TRIGGER도 삭제된다.

Part 3. 데이터베이스 오브젝트 생성하기

Chapter 7. 데이터베이스 오브젝트 생성 계획하기

인덱스

'색인'이라고도 불리며 데이터베이스 객체 중 하나

테이블에 인덱스를 작성

인덱스의 역할은 검색속도의 향상에 있다.

테이블과는 별개의 독립된 객체로 작성되지만 테이블에 의존적이다.

기본키나 유일키와 같은 제약 조건이 지정되면 따로 인덱스를 생성하지 않더라도 해당 키에 대한 인덱스가 자동으로 생성된다.

인덱스의 장점

검색 속도가 빨라진다.

시스템의 부하를 줄여 전체적인 성능을 향상시킨다.

인덱스의 단점

인덱스 생성에 시간이 필요하다.

인덱스를 위한 추가적인 공간이 필요하다.

검색 외 데이터 변경(INSERT / UPDATE / DELETE)가 빈번히 일어날 경우 성능이 저하될 수 있다.

1.1. 인덱스가 필요한 경우

- 테이블의 행의 수가 많을 경우
- WHERE문에 특정 컬럼이 많이 사용될 때
- 검색 결과가 전체 데이터의 2~4%정도일 때
- JOIN에 자주 사용되는 컬럼
- NULL을 포함하는 행이 많은 컬럼일 경우

1.2. 인덱스가 불필요한 경우

- 데이터가 적은(수천 건 미만) 경우
- 조회 보다 삽입, 수정, 삭제 처리가 많은 테이블일 경우
- 조회 결과가 전체 행의 15% 이상 조회할 것으로 예상되는 경우

2. 뷰

물리적인 테이블에 근거한 논리적인 가상의 테이블
데이터가 없고 테이블에 대한 SQL만 저장되어 있다.
사용자가 VIEW에 접근할 경우 SQL이 수행되면서 그 결과를 가져온다.
여러 테이블에 대한 검색결과를 뷰 하나로 검색할 수 있다.
복잡하고 긴 쿼리문을 뷰로 정의하면 접근을 단순화할 수 있다.

2.1. 뷰 문법

```
CREATE [ OR REPLACE ] [ FORCE | NOFORCE ] VIEW view_name
AS subquery
[ WITH CHECK OPTION [CONSTRAINT 제약조건] ]
[ WITH READ ONLY ]
```

CREATE : 뷰 생성
CREATE OR REPLACE : 뷰를 생성하지만 같은 이름의 뷰가 있으면 질의가 변경된다.
FORCE : 기본 테이블 유무에 관계없이 뷰를 생성
WITH CHECK OPTION : 제약조건에 맞는 데이터만 입력되거나 변경될 수 있다.
WITH READ ONLY : SELECT만 가능한 뷰 생성

2.2. 뷰 정보 조회

USER_VIEWS 를 통해 뷰에 대한 정보를 조회 할 수 있다

```
SELECT view_name, text
FROM user_view
WHERE view_name = '뷰이름';
```

2.3. NOFORCE / FORCE

뷰는 일반적으로 기본테이블이 존재한다는 가정 하에 쿼리문을 작성하지만, FORCE 옵션을 이용해서 간혹 존재하지 않는 테이블을 대상으로 뷰를 생성할 수도 있다.

기본값은 NOFORCE이고, 이는 생략가능하다.

3. 시퀀스

테이블 내의 유일한 숫자를 자동으로 생성하는 자동 번호 발생 객체

행을 구분하기 위한 고유 값으로 기본키를 사용한다. 기본키는 중복되지 않은 유일한 값을 가져야 하는데 직접 값을 생성해내는 것이 사용자의 부담이 될 수 있다. 이럴 경우 시퀀스를 이용해 기본키로 지정을 한다.

3.1. 시퀀스 문법

```
CREATE SEQUENCE sequence_name
[ START WITH n ]
[ INCREMENT BY n ]
[ MAXVALUE n | NOMAXVALUE ]
[ MINVALUE n | NOMINVALUE ]
[ CYCLE | NOCYCLE ]
[ CACHE | NOCACHE ]
```

START WITH : 시퀀스 시작 값

INCREMENT BY : 시퀀스 증가 값

MAXVALUE : 시퀀스 최대 값

MINVALUE : 시퀀스 최소 값

CYCLE : 최대값 도달 시 순환 여부, NOCYCLE은 증가완료 시 에러를 유발한다.

CACHE : 메모리상에서 시퀀스 값을 관리(CACHE)할지 여부를 정한다. 기본 값은 20, 원하는 숫자만큼 미리 만들어 캐시로 상주시킨다. NOCACHE설정은 메모리상에서 시퀀스 값을 관리하지 않는다. 캐싱을 하는 것이 퍼포먼스상으로 우위에 있지만, 연속된 번호를 얻어야 한다면 NOCACHE가 바람직하다. (캐싱이 AGING-OUT되거나, DB의 비정상적 종료로 채번이 불연속적으로 될 수 있다. 물론 기본키 상에 번호가 중복될 경우는 없다.)

4. USER

데이터베이스 관리자인 시스템권한(SYS, SYSTEM)은 사용자계정을 발급할 수 있다.

5. ROLE

권한의 부여와 회수를 쉽게 할 수 있게 해준다.

권한의 묶음. 부여하고자 하는 권한들을 ROLE로 묶어 한 번에 여러 권한을 부여하고 회수할 수 있다.

CREATE ROLE 권한을 가진 사용자에 의해 생성 된다.

한 사용자가 여러 ROLE에 접근할 수 있으며, 여러 사용자에게 같은 ROLE을 부여할 수 있다

권한 부여, 회수와 같은 명령어를 사용 한다.

사용자 역할에 맞도록 ROLE을 설정하고 유저에게 지정만 하면 되므로 여러 유저가 있더라도 권한을 부여하는 작업을 효율적으로 처리할 수 있다

6. 동의어(SYNONYM)

시노님은 데이터베이스객체에 대한 별칭이다. 공유하는 테이블에 접근하기 위해서는 **[사용자명.테이블명]**의 문법을 사용하는 데, 이때 동의어를 사용해서 간결하게 할 수 있다.

형식

```
CREATE [PUBLIC] SYNONYM synonym_name
FOR user_name.object_name;
```

object_name은 동의어를 만들려는 데이터베이스객체(테이블, 뷰 등)이다.

6.1. 종류

가. 공개동의어

권한을 주는 사용자가 정의한 동의어로 누구나 사용할 수 있습니다. 동개 동의어는 DBA권한을 가진 사용자만이 생성할 수 있습니다. SYNONYM앞에 PUBLIC을 붙여서 정의한다.

나. 비공개동의어

객체에 대한 접근권한을 부여받은 사용자가 정의한 동의어. 해당사용자만 사용할 수 있다.

6.2. DUAL

DUAL은 SYS가 소유한 테이블로, SYS.DUAL로 접근해야 하지만, SYNONYM을 사용해서 모든 사용자가 DUAL로 간결하게 접근하고 있다.

Chapter 8. 데이터베이스 오브젝트 생성하기

인덱스

1.1. 인덱스 생성

1.1.1. 고유 인덱스(Unique Index)

중복되지 않는 데이터를 인덱스의 키 값으로 지정하는 인덱스

성능이 좋다

인덱스를 생성한 컬럼에 대해서 중복되는 값을 테이블에 삽입하려고 하면 에러 발생

단일 컬럼 고유 인덱스 생성

```
CREATE UNIQUE INDEX index_name ON table_name(column_name);
```

1.1.2. 비고유 인덱스(Non-Unique Index)

중복되는 데이터를 인덱스의 키 값으로 지정하는 인덱스

단일 컬럼 비고유 인덱스 생성

```
CREATE INDEX index_name ON table_name(column_name);
```

1.1.3. 함수 기반 인덱스

인덱스를 사용하려면 WHERE 절의 조건을 다른 형태로 가공해서는 안 된다

만약 WHERE절의 조건으로 'sal + 100'과 같이 가공된 형태로 사용해야 하고 인덱스를 생성하고 싶다면 함수 기반 인덱스를 생성해야 한다.

인덱스의 생성으로 검색 속도를 향상되긴 하지만 데이터베이스 성능향상을 위한 근본적인 해결방안이 아니다. 또한 쿼리 조건이 바뀌면 인덱스를 다시 생성해야하는 단점이 있다.

함수 기반 인덱스 생성

```
CREATE INDEX index_name ON table_name(column_name 가공 형태);
```

함수 기반 인덱스 예

```
CREATE INDEX idx_sal_add_bonus ON emp(sal + 100);
```

1.1.4. 결합 인덱스(Composite Index)

두 개 이상의 컬럼을 합쳐서 인덱스 생성

WHERE 절의 조건 컬럼이 2개 이상 AND로 연결되어 사용되는 경우 많이 사용

결합 인덱스 생성

```
CREATE INDEX index_name ON table_name(column_name1, column_name2, ... );
```

1.1.5. BITMAP 인덱스

데이터 값의 종류가 적고 동일한 데이터가 많을 경우 사용

예) 성별 컬럼 : 남, 여

데이터의 변경양이 적고 값의 종류가 적을 때 좋은 인덱스

데이터가 어디에 존재하는지 정보를 BIT로 표시한다.

BITMAP INDEX 생성 이후 컬럼 값이 새로 하나 더 생긴다면 인덱스를 전부 수정해 줘야 한다.

BITMAP 인덱스 생성

```
CREATE BITMAP INDEX index_name ON table_name(column_name);
```

1.2. 인덱스 제거

```
DROP INDEX index_name;
```

1.3. 인덱스 재생성

대량의 DML 작업을 수행하면 인덱스의 성능이 저하되므로 재설정을 해주는 것이 좋다.

꾸준한 관리가 필요하다.

```
ALTER INDEX index_name REBUILD;
```

1.4. 인덱스 조회

USER_INDEXES 뷰를 이용해 인덱스 정보를 조회할 수 있다.

```
SELECT table_name, index_name, index_type
FROM user_indexes
where table_name = '테이블명';
```

뷰

1.5. VIEW 생성

```
CREATE OR REPLACE FORCE VIEW HUMAN_RESOURCE_VIEW
AS
SELECT EMPNO, ENMAE, DEPTNO
FROM HUMAN_RESOURCE;
```

경고: 컴파일 오류와 함께 뷰가 생성되었습니다.

사용자가 소유한 뷰를 조회해본다

```
SELECT * FROM user_views;
```

	VIEW_NAME	TEXT
1	HUMAN_RESOURCE_VIEW	SELECT EMPNO, ENAME, DEPTNOFROM HUMAN_RESOURCE_VIEW

1.6. VIEW를 통한 DML문 실행

뷰는 가상테이블이지만, 기본테이블을 반영한다. 뷰에서 SELECT문을 통한 데이터 조회 외에도, INSERT/UPDATE/DELETE의 DML문을 실행할 수 있다. 뷰에 대한 DML문은 기본테이블에 적용된다.

EMP테이블의 카피본인 EMP_COPY테이블을 만들고, EMP_COPY테이블에 대한 뷰를 생성, DML문을 실행해보자.

```
--- EMP_COPY 테이블 생성
CREATE TABLE EMP_COPY
AS
SELECT * FROM EMP;

-- VIEW생성
CREATE VIEW EMP_COPY_VIEW
AS
SELECT EMPNO, ENAME, DEPTNO
FROM EMP_COPY;

-- EMP_COPY_VIEW에 INSERT문 실행
```

```
INSERT INTO EMP_COPY_VIEW VALUES(1234,'JONATHAN',30);


--- EMP_COPY_VIEW 조회
SELECT * FROM EMP_COPY_VIEW;
```

결과

	EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
10	7844	TURNER	SALESMAN	7698	81/09/08	1500	0	30
11	7876	ADAMS	CLERK	7788	83/01/12	1100	(null)	20
12	7900	JAMES	CLERK	7698	81/10/03	950	(null)	30
13	7902	FORD	ANALYST	7566	81/10/03	3000	(null)	20
14	7934	MILLER	CLERK	7782	82/01/23	1300	(null)	10
15	1001	ALICE	CLERK	1003	16/09/08	800	(null)	30
16	1002	MORRIS	CLERK	1003	16/09/08	800	(null)	30
17	1003	MATT	SALEMAN	(null)	16/01/01	1500	100	30
18	1010	BOB	(null)	(null)	(null)	(null)	(null)	(null)
19	1011	EDWARD	MANAGER	(null)	(null)	(null)	(null)	(null)
20	1006	JORDAN	MANAGER	7782	82/02/05	5000	(null)	10
21	1234	JONATHAN	(null)	(null)	(null)	(null)	(null)	30

기본테이블인 EMP_COPY을 조회해보자.

```
---EMP_COPY 조회

SELECT * FROM EMP_COPY;
```

결과

	EMPNO	ENAME	DEPTNO
4	7566	JONES	20
5	7654	MARTIN	30
6	7698	BLAKE	30
7	7782	CLARK	10
8	7788	SCOTT	20
9	7839	KING	10
10	7844	TURNER	30
11	7876	ADAMS	20
12	7900	JAMES	30
13	7902	FORD	20
14	7934	MILLER	10
15	1001	ALICE	30
16	1002	MORRIS	30
17	1003	MATT	30
18	1010	BOB	(null)
19	1011	EDWARD	(null)
20	1006	JORDAN	10
21	1234	JONATHAN	30

새로 추가된 'JANATHAN' 사원의 부서번호를 20으로 변경하는 UPDATE문을 실행해보자.

```
-- EMP_COPY_VIEW에 UPDATE문 실행
UPDATE EMP_COPY_VIEW SET DEPTNO=20 WHERE EMPNO=1234;

--- EMP_COPY_VIEW 조회
SELECT * FROM EMP_COPY_VIEW WHERE EMPNO=1234;
```

결과

	EMPNO	ENAME	DEPTNO
1	1234	JONATHAN	20

기본테이블인 EMP_COPY을 조회해보자.

```
---EMP_COPY 조회

SELECT * FROM EMP_COPY WHERE EMPNO=1234;
```

결과

	EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
1	1234	JONATHAN	(null)	(null)	(null)	(null)	(null)	20

'JONATHAN '사원 레코드를 삭제해보자.

```
-- EMP_COPY_VIEW에 DELETE문실행
DELETE FROM EMP_COPY_VIEW WHERE EMPNO=1234;

--- EMP_COPY_VIEW 조회
SELECT * FROM EMP_COPY_VIEW WHERE EMPNO=1234;

---EMP_COPY 조회
SELECT * FROM EMP_COPY WHERE EMPNO=1234;

---EMP_COPY 조회
SELECT * FROM EMP_COPY WHERE EMPNO=1234;
```

EMP_COPY테이블에서 'JONATHAN'사원이 삭제되고, 이를 참조하는 뷰인 EMP_COPY_VIEW에서도 'JONATHAN '사원이 보이지 않는다.

1.7. WITH CHECK OPTION

뷰에 대한 DML문을 실행할 때 뷰 생성 시 조건제시에 사용된 컬럼 값을 변경하지 못하도록 한다.

부서번호가 30인 사원들만을 열람하는 뷰 EMP_VIEW_30을 WITH CHECK OPTION으로 생성하고,

```
-- VIEW생성
CREATE VIEW EMP_VIEW_30
AS
SELECT EMPNO, ENAME, DEPTNO
FROM EMP_COPY
WHERE DEPTNO = 30
WITH CHECK OPTION;

--- EMP_VIEW_30조회
SELECT * FROM EMP_VIEW_30;
```

결과

	EMPNO	ENAME	DEPTNO
1	7499	ALLEN	30
2	7521	WARD	30
3	7654	MARTIN	30
4	7698	BLAKE	30
5	7844	TURNER	30
6	7900	JAMES	30
7	1001	ALICE	30
8	1002	MORRIS	30
9	1003	MATT	30
10	1234	JONATHAN	30

JONATHAN 사원의 부서번호를 20으로 변경해보자.

```
UPDATE EMP_VIEW_30 SET DEPTNO=20

WHERE EMPNO=1234;
```

```
오류 보고 -

ORA-01402: view WITH CHECK OPTION where-clause violation
```

위와 같이 WITH CHECK OPTION설정으로 뷰 생성 시 조건으로 제시된 컬럼을 변경할 수 없게 된다.

1.8. WITH READ ONLY

뷰에 가진 모든 컬럼에 대하여 UPDATE를 할 수 없도록 방지하는 것이다.
WITH CHECK OPTION이 뷰 생성 시 조건 컬럼에 대해서만 UPDATE를 방지하는 것과 달리, 뷰의 모든 컬럼의 정보를 수정할 수 없게 된다.

```
-- VIEW생성
CREATE VIEW EMP_VIEW_30
AS
SELECT EMPNO, ENAME, DEPTNO
FROM EMP_COPY
WHERE DEPTNO = 30
WITH READ ONLY;

--- JONATHAN사원 이름 변경
UPDATE EMP_VIEW_30 SET ENAME='JAKE'
WHERE EMPNO=1234;
```

```
SQL 오류: ORA-42399: cannot perform a DML operation on a read-only view

42399.0000 - "cannot perform a DML operation on a read-only view"
```

위와 같이 EMP_VIEW_30의 컬럼의 정보를 수정하려고 하자, READ-ONLY VIEW에서는 DML문을 처리할 수 없다는 오류가 발생하고, 사용자는 정보를 갱신할 수 없다.

1.9. TOP-N 구하기

TOP-N은 예를 들어, 연봉이 높은 상위5명의 사원을 구한다거나, 입사일이 빠른 10명의 사원을 구하는 데 사용된다.

ROWNUM은 오라클에서 내부적으로 부여하는 컬럼이다. 데이터베이스에 INSERT된 순서대로 카운팅 된다.

```
SELECT ROWNUM, EMPNO, ENAME, SAL
FROM EMP;
```

결과

	ROWNUM	EMPNO	ENAME	SAL
1	1	7369	SMITH	800
2	2	7499	ALLEN	1600
3	3	7521	WARD	1250
4	4	7566	JONES	2975
5	5	7654	MARTIN	1250
6	6	7698	BLAKE	2850
7	7	7782	CLARK	2450
8	8	7788	SCOTT	3000
9	9	7839	KING	5000
10	10	7844	TURNER	1500
11	11	7876	ADAMS	1100
12	12	7900	JAMES	950
13	13	7902	FORD	3000
14	14	7934	MILLER	1300
15	15	1001	ALICE	800
16	16	1002	MORRIS	800
17	17	1003	MATT	1500
18	18	1010	BOB	(null)
19	19	1011	EDWARD	(null)
20	20	1006	JORDAN	5000

위 예제와 같이 별도의 ORDER BY 절 기술하지 않는다면, 내부적으로 ROWNUM의 순서에 따라 정렬하게 된다.

ROWNUM은 내부적으로 이미 부여받은 값이기 때문에 ORDER BY 절을 통해 순서를 바꾸더라도 값은 변하지 않는다.

```
SELECT ROWNUM, EMPNO, ENAME, SAL
FROM EMP
ORDER BY SAL DESC;
```

결과

	ROWNUM	EMPNO	ENAME	SAL
1	20	1006	JORDAN	5000
2	9	7839	KING	5000
3	13	7902	FORD	3000
4	8	7788	SCOTT	3000
5	4	7566	JONES	2975
6	6	7698	BLAKE	2850
7	7	7782	CLARK	2450
8	2	7499	ALLEN	1600
9	10	7844	TURNER	1500
10	17	1003	MATT	1500
11	14	7934	MILLER	1300
12	3	7521	WARD	1250
13	5	7654	MARTIN	1250
14	11	7876	ADAMS	1100
15	12	7900	JAMES	950
16	16	1002	MORRIS	800
17	1	7369	SMITH	800
18	15	1001	ALICE	800
19	19	1011	EDWARD	(null)
20	18	1010	BOB	(null)

뷰와 ROWNUM을 이용해 TOP-N 구하기

```
-- VIEW생성
CREATE OR REPLACE VIEW EMP_VIEW_SAL
AS
SELECT EMPNO, ENAME, NVL(SAL,0) SAL
FROM EMP
ORDER BY SAL DESC;
--- EMP_VIEW_SAL 조회
SELECT ROWNUM, EMPNO, ENAME, SAL
FROM EMP_VIEW_SAL;
```

결과

	ROWNUM	EMPNO	ENAME	SAL
1	1	1006	JORDAN	5000
2	2	7839	KING	5000
3	3	7902	FORD	3000
4	4	7788	SCOTT	3000
5	5	7566	JONES	2975
6	6	7698	BLAKE	2850
7	7	7782	CLARK	2450
8	8	7499	ALLEN	1600
9	9	7844	TURNER	1500
10	10	1003	MATT	1500
11	11	7934	MILLER	1300
12	12	7521	WARD	1250
13	13	7654	MARTIN	1250
14	14	7876	ADAMS	1100
15	15	7900	JAMES	950
16	16	1002	MORRIS	800
17	17	7369	SMITH	800
18	18	1001	ALICE	800
19	19	1011	EDWARD	0
20	20	1010	BOB	0

위 예제에서 뷰를 생성하면서, ROWNUM 컬럼 값이 새롭게 부여받았다. 이를 이용해서 월급인 높은 상위 5명을 구해보자.

```
SELECT ROWNUM, EMPNO, ENAME, SAL
FROM EMP_VIEW_SAL
WHERE ROWNUM <=5;
```

결과

	ROWNUM	EMPNO	ENAME	SAL
1	1	7839	KING	5000
2	2	1006	JORDAN	5000
3	3	7788	SCOTT	3000
4	4	7902	FORD	3000
5	5	7566	JONES	2975

위 예제에서 WHERE절에 ROWNUM이 5이하인 사원만 추려내어 출력하였다.

인라인뷰로 TOP-N구하기

위 예제를 인라인뷰를 통해서 구해보자. 인라인뷰는 1회성의 서브쿼리이다. 보통 FROM절에서 사용하며, 테이블처럼 취급하면 된다.

형식

```
SELECT ...
FROM ... ( SELECT ...
           FROM ...) --서브쿼리
); --메인쿼리
```

```
SELECT ROWNUM, EMPNO, ENAME, SAL
FROM ( SELECT EMPNO, ENAME, NVL(SAL,0) SAL
       FROM EMP
       ORDER BY SAL DESC )
WHERE ROWNUM <= 5;
```

결과

	ROWNUM	EMPNO	ENAME	SAL
1	1	7839	KING	5000
2	2	1006	JORDAN	5000
3	3	7788	SCOTT	3000
4	4	7902	FORD	3000
5	5	7566	JONES	2975

뷰를 명시적으로 생성하지 않고, 인라인뷰를 활용하여 월급 상위5명의 명단을 구할 수 있다. 메인쿼리 안쪽에 인라인뷰를 정의하고, 이때 생성된 ROWNUM컬럼을 이용하여 상위 5명을 추려낸다.

2. 시퀀스

2.1. 시퀀스 생성 및 활용

```
CREATE SEQUENCE seq_empno
START WITH 1000
INCREMENT BY 1
CACHE 20;
```

```
SELECT seq_empno.NEXTVAL FROM dual;
SELECT seq_empno.NEXTVAL FROM dual;
SELECT seq_empno.NEXTVAL FROM dual;
```

```
1000
1001
1002
```

INSERT문에서 시퀀스 사용

```
INSERT INTO emp (empno, ename)
VALUES (seq_empno.nextval, 'TEST');
```

```
SELECT empno, ename FROM emp WHERE ename = 'TEST';
```

```
EMPNO      ENAME
----------  ----------
1004       TEST
```

2.2. 시퀀스 수정

START WITH 값을 제외한 다른 설정들을 수정할 수 있다.

```
ALTER SEQUENCE sequence_name
[ INCREMENT BY n ]
[ MAXVALUE n | NOMAXVALUE ]
[ MINVALUE n | NOMINVALUE ]
[ CYCLE | NOCYCLE ]
[ CACHE | NOCACHE ];
```

시퀀스 수정 예

```
ALTER SEQUENCE seq_empno
INCREMENT BY 10
CACHE 40;
```

```
SELECT seq_empno.NEXTVAL FROM dual;
SELECT seq_empno.NEXTVAL FROM dual;
SELECT seq_empno.NEXTVAL FROM dual;
```

```
1010
1024
1034
```

2.3. 시퀀스 삭제

DROP SEQUENCE 문으로 시퀀스를 삭제한다.

```
DROP SEQUENCE seq_empno;
```

2.4. 시퀀스 DATA DICTIONARY

사용자의 시퀀스객체 정보를 저장하고 있는 USER_SEQUENCES 테이블을 조회해보자.

```
SELECT SEQUENCE_NAME, MIN_VALUE, MAX__VALUE, INCREMENT_BY, CYCLE_FLAG,
LAST_NUMBER
FROM USER_SEQUENCES;
```

결과

	SEQUENCE_NAME	MIN_VALUE	MAX_VALUE	INCREMENT_BY	CYCLE_FLAG	ORDER_FLAG	CACHE_SIZE	LAST_NUMBER
1	SEQ_EMPNO	1	999999999999999999999999999	1	N	N	20	1020

이중 LAST_NUMBER 컬럼의 값이 마지막 숫자 값을 나타내며, 사용자가 CACHE설정을 해두었다면, CACHE에 의해 채번된 마지막 번호 값을 나타낸다.

3. USER

3.1. 사용자계정 생성

관리자계정으로 접속해서 새로운 사용자 USER_NEW를 생성해보자.

```
--관리자계정 접속
CONN sys/oracle AS SYSDBA;


--USER_NEW 생성
--사용자비밀번호 입력 : USER_NEW_PW
CREATE USER USER_NEW IDENTIFIED BY USER_NEW_PW;


--사용자생성확인
SELECT * FROM DBA_USERS;
```

결과

	USERNAME	USER_ID	PASSWORD	ACCOUNT_STATUS	LOCK_DATE	EXPIRY_DATE	DEFAULT_TABLESPACE	TEMPORARY_TABLESI
1	USER_NEW	50	(null)	OPEN	(null)	18/03/04	SYSTEM	TEMP
2	SCOTT	48	(null)	OPEN	(null)	18/02/24	SYSTEM	TEMP
3	TEST_USER1	49	(null)	OPEN	(null)	18/03/03	SYSTEM	TEMP
4	SYS	0	(null)	OPEN	(null)	18/02/24	SYSTEM	TEMP
5	SYSTEM	5	(null)	OPEN	(null)	18/02/24	SYSTEM	TEMP
6	ANONYMOUS	35	(null)	OPEN	(null)	14/11/25	SYSAUX	TEMP
7	APEX_PUBLIC_USER	45	(null)	LOCKED	14/05/29	14/11/25	SYSTEM	TEMP
8	FLOWS_FILES	44	(null)	LOCKED	14/05/29	14/11/25	SYSAUX	TEMP
9	APEX_040000	47	(null)	LOCKED	14/05/29	14/11/25	SYSAUX	TEMP

사용자를 생성하더라도, 기본권한인 CREATE SESSION이 없으면, 접속할 수 없다. CREATE SESSION을 포함한 기본 롤(권한묶음) CONNECT와 RESOURCE를 부여한다.

```
--기본권한 부여
GRANT CONNECT, RESOURCE TO USER_NEW;


--사용자 USER_NEW로 접속

CONN USER_NEW/USER_NEW_PW;
```

3.2. 기본적인 롤

ROLE	포함하는 권한
CONNECT	ALTER SESSION, CREATE CLUSTER, CREATE DATABASE LINK, CREATE SEQUENCE, CREATE SESSION, CREATE SYNONYM, CREATE TABLE, CREATE VIEW
RESOURCE	CREATE CLUSTER, CREATE PROCEDURE, CREATE SEQUENCE, CREATE TABLE, CREATE TRIGGER
DBA	DATABASE객체 관리, 사용자계정관련 모든 권한.

3.3. 사용자계정 삭제

```
--관리자계정 접속
CONN sys/oracle AS SYSDBA;

--USER_NEW 계정삭제
DROP USER USER_NEW;
```

사용자계정을 삭제하면, 사용자 관련 권한, 롤 모두 일괄적으로 삭제된다.

4. ROLE

4.1. ROLE 생성 문법

```
CREATE ROLE role_name
```

4.2. ROLE을 이용한 사용자의 권한 처리 예

```
-- ROLE 생성
SQL> CREATE ROLE role_manager;

-- ROLE에 권한 부여
SQL> GRANT create session, create table TO role_manager;

-- scott, test1, test2 사용자에게 ROLE 부여
SQL> GRANT role_manager TO scott, test1, test2;
```

4.3. ROLE을 부여한 사용자의 권한 회수

```
SQL> REVOKE role_manager FROM test1, test2;
```

5. 동의어(SYNONYM)

6.1. 공개동의어 생성

공개동의어를 생성해보자. 공개동의어는 DBA가 정의하고 모든 사용자가 열람할 수 있다.
SYS계정으로 로그인해서 SCOTT계정의 EMP테이블의 공개동의어를 생성한다.

```
--- SYS로 로그인.
CONN SYS/ORACLE AS SYSDBA;
CREATE PUBLIC SYNONYM EMP_PUBLIC
FOR SCOTT.EMP;

--- 새로운 테스트유저를 생성하고, 기본롤과 해당테이블 열람권한을 부여하자.
CREATE USER TEST_USER1 IDENTIFIED BY 1234;
GRANT CONNECT, RESOURCE TO TEST_USER1;
GRANT SELECT ON SCOTT.EMP TO TEST_USER1;

--- TEST_USER1로 접속하고, EMP_PUBLIC에 접근해보자.
CONN TEST_USER1/1234;
SELECT * FROM EMP_PUBLIC;
```

결과

	EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
1	7369	SMITH	CLERK	7902	16/12/19	800	(null)	20
2	7499	ALLEN	SALESMAN	7698	81/02/20	1600	300	30
3	7521	WARD	SALESMAN	7698	81/02/22	1250	200	30
4	7566	JONES	MANAGER	7839	81/04/02	2975	30	20
5	7654	MARTIN	SALESMAN	7698	81/09/28	1250	300	30
6	7698	BLAKE	MANAGER	7839	81/04/01	2850	(null)	30
7	7782	CLARK	MANAGER	7839	81/06/01	2450	(null)	10
8	7788	SCOTT	ANALYST	7566	82/10/09	3000	(null)	20
9	7839	KING	PRESIDENT	(null)	81/11/17	5000	3500	10
10	7844	TURNER	SALESMAN	7698	81/09/08	1500	0	30
11	7876	ADAMS	CLERK	7788	83/01/12	1100	(null)	20
12	7900	JAMES	CLERK	7698	81/10/03	950	(null)	30
13	7902	FORD	ANALYST	7566	81/10/03	3000	(null)	20
14	7934	MILLER	CLERK	7782	82/01/23	1300	(null)	10
15	1001	ALICE	CLERK	1003	16/09/08	800	(null)	30
16	1002	MORRIS	CLERK	1003	16/09/08	800	(null)	30
17	1003	MATT	SALEMAN	(null)	16/01/01	1500	100	30
18	1010	BOB	(null)	(null)	(null)	(null)	(null)	(null)
19	1011	EDWARD	MANAGER	(null)	(null)	(null)	(null)	(null)
20	1006	JORDAN	MANAGER	7782	82/02/05	5000	(null)	10

위 예제에서 원칙적으로는 SCOTT.EMP로 접근해야 하지만, SYNONYM인 EMP_PUBLIC을 사용해서 간결하게 접근할 수 있다.

6.2. 비공개동의어 생성

비공개 동의어를 생성해보자. 비공개동의어는 정의한 사용자만 사용 가능하다.

```
--- TEST_USER1로 접속하고, 비공개동의 MY_EMP를 만들어보자.
CONN TEST_USER1/1234;
CREATE SYNONYM MY_EMP
FOR SCOTT.EMP;

--MY_EMP 동의어로 테이블접근.
SELECT * FROM MY_EMP;
```

결과

	EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
1	7369	SMITH	CLERK	7902	16/12/19	800	(null)	20
2	7499	ALLEN	SALESMAN	7698	81/02/20	1600	300	30
3	7521	WARD	SALESMAN	7698	81/02/22	1250	200	30
4	7566	JONES	MANAGER	7839	81/04/02	2975	30	20
5	7654	MARTIN	SALESMAN	7698	81/09/28	1250	300	30
6	7698	BLAKE	MANAGER	7839	81/04/01	2850	(null)	30
7	7782	CLARK	MANAGER	7839	81/06/01	2450	(null)	10
8	7788	SCOTT	ANALYST	7566	82/10/09	3000	(null)	20
9	7839	KING	PRESIDENT	(null)	81/11/17	5000	3500	10
10	7844	TURNER	SALESMAN	7698	81/09/08	1500	0	30
11	7876	ADAMS	CLERK	7788	83/01/12	1100	(null)	20
12	7900	JAMES	CLERK	7698	81/10/03	950	(null)	30
13	7902	FORD	ANALYST	7566	81/10/03	3000	(null)	20
14	7934	MILLER	CLERK	7782	82/01/23	1300	(null)	10
15	1001	ALICE	CLERK	1003	16/09/08	800	(null)	30
16	1002	MORRIS	CLERK	1003	16/09/08	800	(null)	30
17	1003	MATT	SALEMAN	(null)	16/01/01	1500	100	30
18	1010	BOB	(null)	(null)	(null)	(null)	(null)	(null)
19	1011	EDWARD	MANAGER	(null)	(null)	(null)	(null)	(null)
20	1006	JORDAN	MANAGER	7782	82/02/05	5000	(null)	10

위와 같이 MY_EMP라는 동의어로 SCOTT.EMP라는 테이블에 접근할 수 있다.

MY_EMP라는 동의어를 SCOTT계정에서 사용가능하지 확인해보자.

```
--- SCOTT으로 접속하고, 비공개동의 MY_EMP로 접속해보자.
CONN SCOTT/TIGER;
SELECT * FROM MY_EMP;
```

```
ORA-00942: table or view does not exist
```

테이블의 소유주인 SCOTT일지라도, TEST_USER1이 생성한 비공개동의어 MY_EMP로는 본인의 EMP테이블에 접근할 수 없다.

Chapter 9. 공공데이터 모델링하기

1.1 공공데이터 제공 사이트 접속하기

- 정부에서 제공하는 공공데이터 포털사이트를 이용하면 공공데이터를 손쉬게 이용할 수 있다. 브라우저를 열고 https://www.data.go.kr/index.do접속하여 정부에서 제공하는 공공데이터를 활용할 수 있다.



중간에 있는 검색창을 이용해서 원하는 데이터를 검색할 수가 있고 세부검색조건을 아래 있는 상세 조건으로 검색할 수 있다.

1.2 공공데이터 회원가입하기

일단 공공데이터포털에서 제공하는 공공데이터를 이용하려면 회원가입을 해야한다.



1. 이름과 사용이 가능한 email을 입력하고 가입확인을 한다. -> 가입정보가 없음 출력

2. 연력에 맞춰 가입하기를 선택한다.

3. 각 항목에 상세 정보를 입력하여 회원가입을 완료한다.

1.3 공공데이터 조회하기

필요한 데이터는 공공데이터 포털 사이트의 메인화면 조회창을 이용해서 검색할수도 있고 인기데이터, 최신데이터 등 링크로 걸려있는 추천 데이터를 확인할 수 있다.



- 조회창 이용해서 데이터 검색하기

원하는 데이터 내용을 조회창에 검색을 하면 검색 결과가 나오고 검색결과는 파일데이터, 오픈API로 구분하여 확인할 수 있다. 파일데이터는 데이터가 파일로 저장되어있는 형태를 말하는것이고 오픈 API는 요청주소로 HttpRequest를 보내서 받아오는 방식을 의미한다. 검색 조건을 상세화하여 필터링도 가능하다.



- 파일데이터 상세보기

파일데이터 상세보기를 하면 데이터가 파일을 다운로드할 수 있는 버튼이 있고, 데이터 관련 상세내용을 확인할 수 있다. 미리보기를 보면 데이터의 구조와 내용을 미리 확인할 수 있다. 파일로 제공되는 공공데이터는 별로 인증키 없이 다운로드가 가능하다



파일데이터 상세

CSV 경기도 구리시_전기차충전소

구리시 전기차충전소 정보

0 0 관심

다운로드

오류신고 및 담당자 문의

파일데이터 정보 메타데이터 다운로드

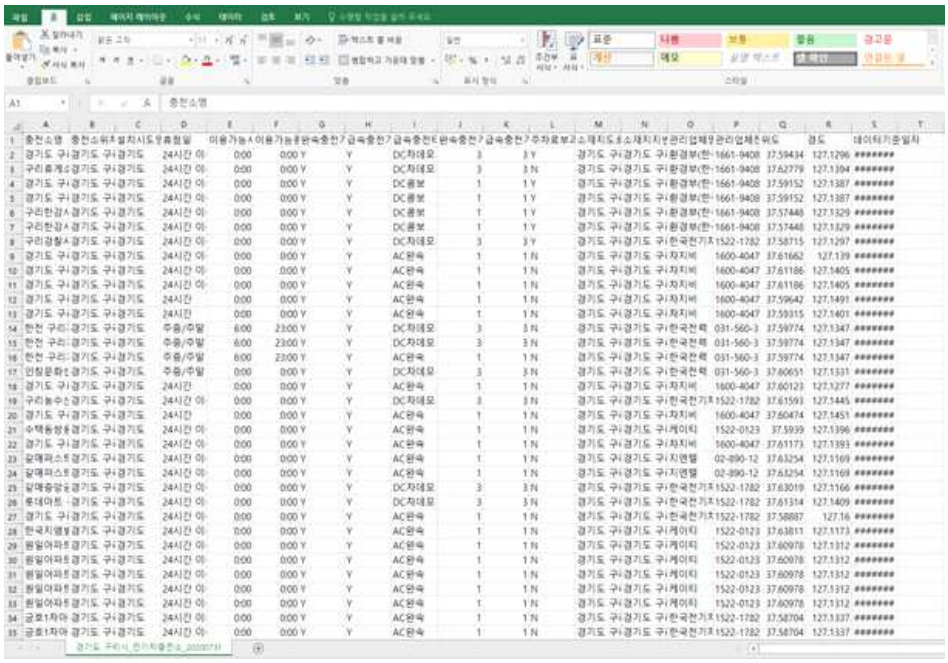
파일데이터명	경기도 구리시_전기차충전소_20200731		
분류체계	산업 통상·중소기업 - 에너지및자원개발	제공기관	경기도 구리시
관리부서명	정보통신과	관리부서 전화번호	031-550-2087
보유근거		수집방법	
업데이트 주기	연간	자기 등록 예정일	2021-07-31
매체유형	텍스트	전체 행	44
확장자	CSV	다운로드(바로가기)	244
데이터 한계		키워드	전기차,전기차충전,구리시
등록	2020-11-09	수정	2020-11-09
제공형태	공공데이터포털에서 다운로드(원문파일등록)		
설명	구리시 전기차충전소 정보		
기타 유의사항			
비용부과유무	무료	비용부과기준 및 단위	건
이용허락범위	이용허락범위 제한 없음		

미리보기

※ 파일 데이터의 일부 내용을 제공하고 있으며, 전체 내용이 필요한 경우 해당 파일을 다운로드 받으시기 바랍니다.

충전소명	충전소위치상세	설치시도명	휴정일	이용가능시작시각	이용가능종료시각	완속충전가능여부	급속충전가능여부	급속충전타입구분	완속충전기대수
경기도 구리시청	경기도 구리시 아차산로 439 본관 흡연실 옆	경기도	24시간 이용가능	00:00	00:00	Y	Y	DC차데모+AC3상+DC콤보	3
구리홍계소(외곽방향)	경기도 구리시 서울외곽순환고속도로 32 (사노동)	경기도	24시간 이용가능	00:00	00:00	Y	Y	DC차데모+AC3상+DC콤보	3
경기도 구리시 청정복지센터	경기도 구리시 체육관로 74	경기도	24시간 이용가능	00:00	00:00	Y	Y	DC콤보	1
경기도 구리시 청정복지센터	경기도 구리시 체육관로 74	경기도	24시간 이용가능	00:00	00:00	Y	Y	DC콤보	1
구리한강시민공원(주차장)	경기도 구리시 코스모스길14번길 249	경기도	24시간 이용가능	00:00	00:00	Y	Y	DC콤보	1
구리한강시민공원(주차장)	경기도 구리시 코스모스길14번길 249	경기도	24시간 이용가능	00:00	00:00	Y	Y	DC콤보	1
구리경찰서	경기도 구리시 아차산로 359	경기도	24시간 이용가능	00:00	00:00	Y	Y	DC차데모+AC3상+DC콤보	3

미리보기에 있는 데이터구조와 내용을 보고 데이터 저장방식 및 class를 설계하면 된다. csv파일을 다운로드 받으면 엑셀로 내부 데이터 전부를 확인할 수 있음.



csv의 엑셀내용을 참고 하여 DB테이블에 대한 모델링을 하면 된다. 오라클 디벨로퍼를 이용하면 csv방식의 데이터를 바로 테이블로 전환할 수 있다.

1.4 공공데이터 파일이용해서 테이블 만들기

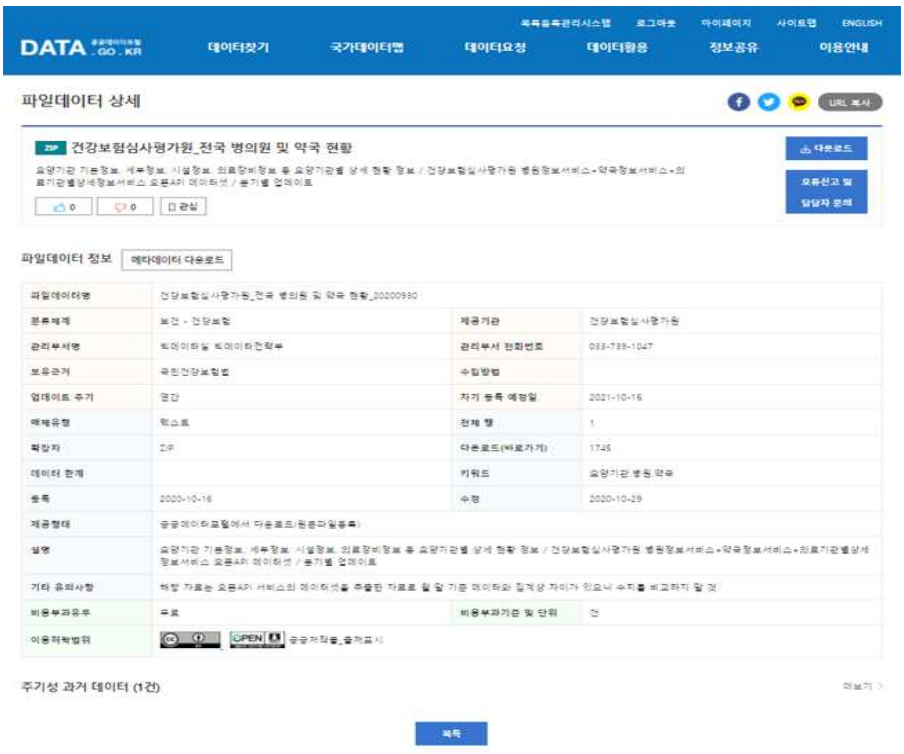
파일방식으로 제공하는 데이터를 검색하여 오라클 데이터 베이스에 테이블로 생성해보자.

- csv방식의 공공데이터 선정하기

공공데이터 포털에서 csv방식으로 제공하는 데이터를 검색하여 다운로드 하기 검색창에 "건강보험 심사평가원_전국 병의원 및 약국현황"입력하여 조회해보자.



결과 화면에서 첫 번째 데이터의 상세화면으로 들어가서 파일을 다운로드 한다. 파일은 ZIP파일로 제공하고 있다.



ZIP파일을 다운받아 압축을 해제하면 아래 그림과 같이 여러 가지 csv파일을 볼 수가 있다. 상세화면은 클릭을 하면 엑셀파일로 편하게 볼 수 있고 이 파일을 가지고 간단하게 DB에 테이블로 전환하여 사용할 수 있다.

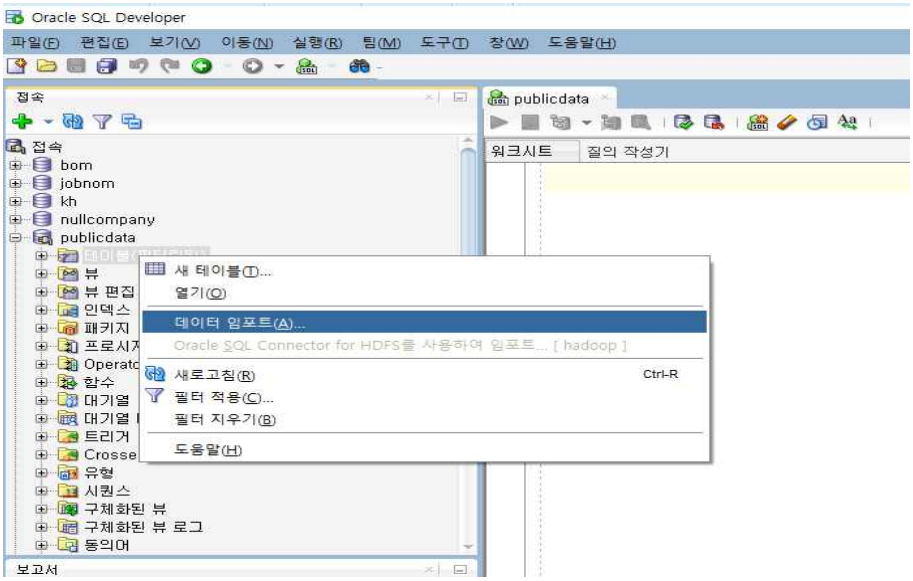


- sql developer이용하여 테이블 만들기

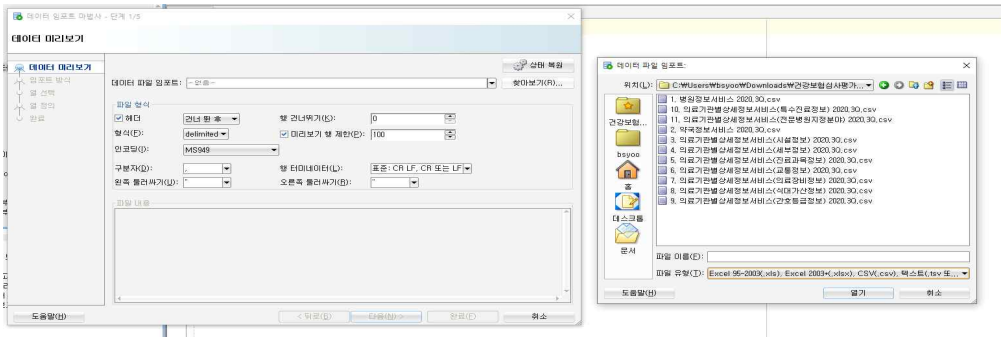
사용자 계정 생성하기 system 계정으로 공공데이터 모델링할 사용자 생성 및 권한부여



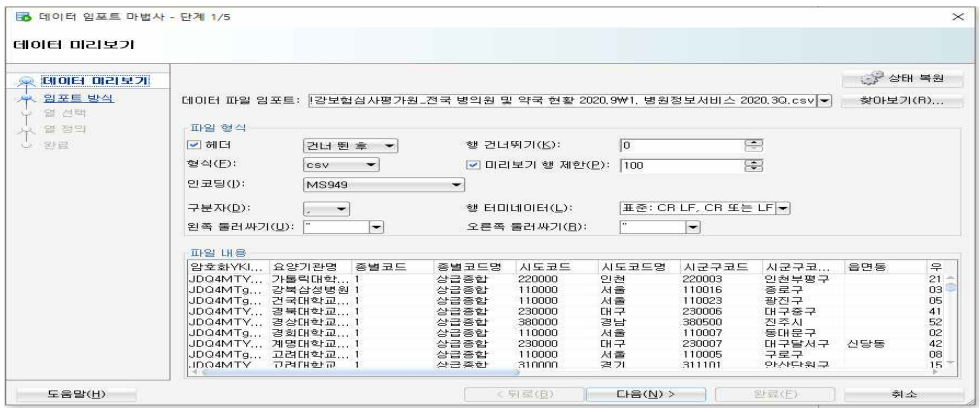
계정으로 접속 후 테이블 폴더에서 데이터 임포트 선택



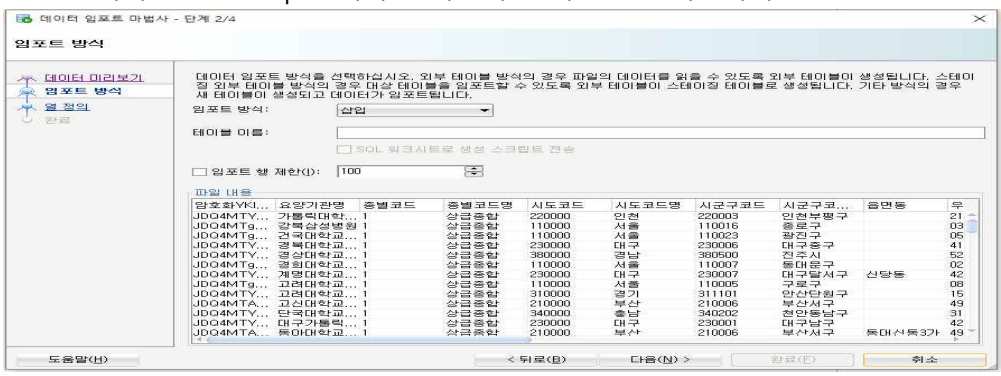
팝업창에서 공공데이터에서 받아온 csv중 병원정보서비스를 테이블로 만들어 보자.



파일을 선택하면 csv의 데이터가 DB의 테이블 형식으로 들어온 것을 확인 할 수가 있다. 다음버튼으로 이동하자.

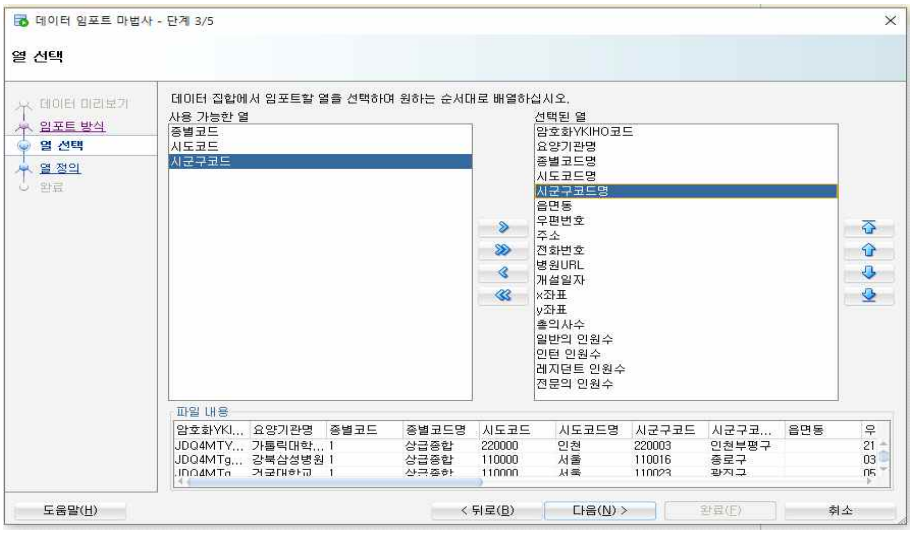


다음버튼으로 이동을 하게 되면 테이블이름을 설정할 수 있다. 테이블명을 hospital이라고 작성하고 다음으로 이동하자.

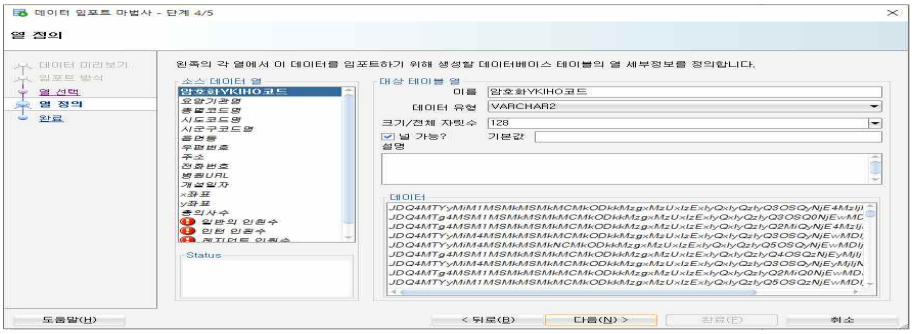


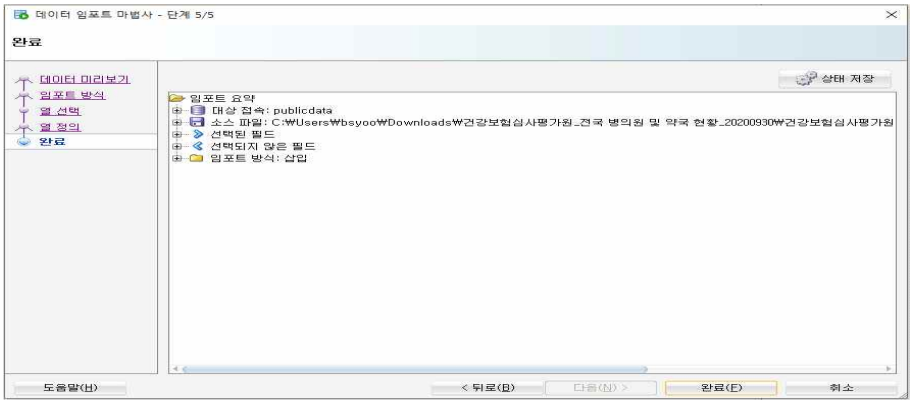
다음으로 이동하면 csv에서 필요한 항목만 선택해서 테이블을 생성할 수가 있다. 필요 없는 코드는 빼도록 하자. * 필요에 의해 조정이

가능하다.



항목별 데이터의 자료형, 컬럼명을 설정할 수 있다. 컬럼명을 영어로 변환하여 저장하도록 하자.



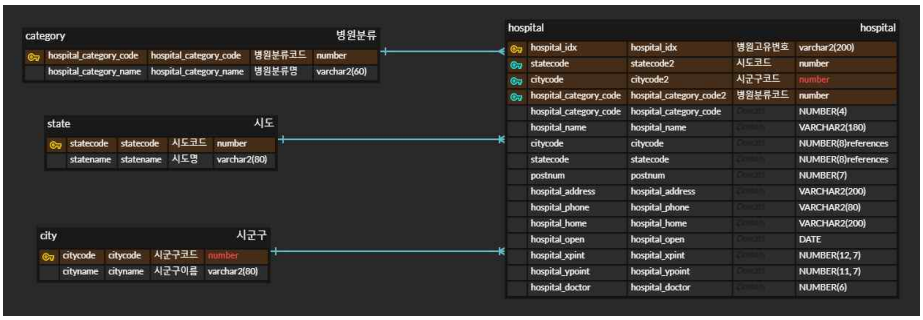


완료를 하면 해당 csv에 있는 데이터가 테이블로 옮겨진다.

공공데이터 포털에서 받아온 csv, excel형태의 자료를 오라클에 넣어서 활용할 수가 있다. 공공데이터를 테이블로 변환하여 저장한 뒤 웹 애플리케이션에서 사용할 테이블과 제약조건을 설정해서 사용할 수 가 있다. 공공데이터를 이용하면 데이터 자료를 가져와 웹 애플리케이션에 다양하게 적용하고 DB을 설계할 수 있다.

- 공공데이터 분석하여 모델링하기

csv데이터를 보고 정규화를 간단하게 해보면 병원정보 데이터에서 아래와 같이 모델링하여 테이블을 나누어 저장할 수 있다.



csv를 가져온 데이터 구조를 분석하고 정규화를 통해 적절하게 모델링하여 DB를 구성할 수 있다.
모델링대로 csv파일 테이블로 구성한 다음 제약조건을 설정하면 된다.
공공데이터 편집하기 카테고리 목록으로 만들기 위해 데이터는 편집하는 과정으로 카테로리 테이블에 들어갈 값에 중복 값을 없애는 과정이다. 중복값을 제거한 데이터를 임시 테이블(temp)을 만들어서 넣고 기존 테이블에 다시 삽입하는 방식으로 사용하였다. 그 후 카테로리테이블들과 hospital테이블 연결을 위해 제약조건(primary key, foreign key)를 설정하였다.
테이블 구성



```
create table temp(
    code number,
    name varchar2(100)
);
insert into temp
select distinct HOSPITAL_CATEGORY_CODE, HOSPITAL_CATEGORY_NAME from category;
delete from category;
insert into category
select * from temp;
select * from category;
delete from temp;
insert into temp
select distinct stateCODE, stateNAME from state;
select * from temp;
delete from state;
insert into state
select * from temp;
select * from state;
delete from temp;
insert into temp
select distinct citycode,cityname from city;
delete from city;
insert into city
select * from temp;
```

제약조건 설명 명령어 실행

```
alter table hospital add constraint pk_idx primary key(hospital_idx);
alter table category add constraint pk_cateory primary key(hospital_category_code);
alter table state add constraint pk_state primary key(statecode);
alter table city add constraint pk_city primary key(citycode);
alter table hospital add constraint fk_category foreign key(hospital_category_code) references category(hospital_category_code);
alter table hospital add constraint fk_state foreign key(statecode) references state(statecode);
alter table hospital add constraint fk_city foreign key(citycode) references city(citycode);
```

DB모델링한대로 DB를 구축해보았다 이처럼 공공데이터를 가져와서 모델링 후 각종 애플리케이션에 적용하여 더 많은 기능을 수행하게 할 수 있다.